Alexandria Higher Institute of Engineering & Technology (AIET)

Program of Electronics and Communication Engineering (ECE)

Final Year Project Report (2025-2026)

Integrated Software Engineering Solutions

Students' Names

Ali Ayman Mohamed Gamal Eldeen	ECE
Mohamed Elshazly AbdElaziz Mobarak	ECE
Kyrellos Hany Saied Fahem	ECE
Marwan Samy Mostafa Elzwawy	CE
Rewan Alaa Abd Elmaksoud	ECE
Eman Ahmed Ghaith	ECE
Alaa Reda Mohammad Abo Mehana	ECE
Rowaida Ahmed Abd El-Wahab	ECE
Aya Ahmed Khattab	ECE
Mohamed Ahmed Mohamed Aly	ECE

A report submitted in part fulfilment of the degree of

BSc in Program of Electronics and Communication Engineering (ECE)

Supervisors

Dr. Ola Hussein Dr. Fatma Ahmed

Head of Department

Dr. Mohamed El-Amir Attalla

June 22, 2026

Implementation of the Egypt's 2030 sustainable Development goals (sdgs) through aiet's projects

Alexandria Higher Institute of Engineering &Technology (AIET)

Professor kassim kadry (Dean of AIET)

What is Sustainable Development?

"Development that meets the needs of the present without compromising the ability of future generations to meet their own needs."

In other words, Sustainable Development is the criteria for achieving social and economic progress in ways that will not exhaust the earth's finite resources and not exploit or impoverish one grouping of people for the enrichment of another.

What are the world's 2030 Sustainable Development Goals?

On September 25, 2015, after three years of debate and negotiations, all 193 member-nations of the UN, voted unanimously to adopt these global Sustainable Development Goals (SDG's), stating:

"On behalf of the peoples we serve, we have adopted a historic decision on a comprehensive, far-reaching, and people-centered set of universal and transformative goals and targets. We commit ourselves to working tirelessly for the full implementation of this agenda by 2030."

How are the SDGs supposed to be implemented?

- "…all member states to develop as soon as practicable, ambitious national responses to the overall implementation of this Agenda. These can…build on existing planning instruments, such as national development and sustainable development strategies, as appropriate."

- "…the essential role of national parliaments through their enactment of legislation and adoption of budgets and their role in insuring accountability for the effective implementation of our commitments."

- "…governments and public institutions…work closely on implementation with regional and local authorities, sub-regional institutions, international institutions,academia, philanthropic organizations, volunteer groups and others."

How will the progress or lack of progress be monitored?

A. Voluntary National Reviews at the UN

Each country is encouraged to"…conduct regular and inclusive reviews of progress at the national and sub-national levels, which are country-led and country-driven." "Regular reviews at the HLPF are to be voluntary, state-led,undertaken by both developed and developing countries, and involve multiple stakeholders."

B. Stakeholders. That's YOU! We, the people affected most by the need for these Goals must keep our government accountable to the 2030 Sustainable Development Goals!

To do that, we must organize!

Egypt Vision 2030

Egypt Vision 2030 focuses on improving the life quality and living standard of the Egyptian citizen in various aspects of life through emphasizing the consolidation of justice principles and social integration, and the participation of all citizens in the political and social life. This comes hand in hand with achieving high, inclusive and sustainable economic growth, promoting investment in people and building their creative capabilities by encouraging the increase of knowledge, innovation and scientific research in all fields. The Vision also focuses on the governance of the State's institutions and society through the administrative reform, consolidating transparency, supporting monitoring and evaluation systems, and empowering local administrations. All of these desired goals come within the framework of ensuring the peace and security of Egypt and promoting the Egyptian leadership regionally and internationally.

INTRODUCTION

Inspired by the ancient Egyptian Civilization, linking the present to future, the Sustainable Development Strategy (SDS): Egypt Vision 2030 represents a foothold on the way towards inclusive development. Thus, cultivating a prosperity path through, economic and social justice, and reviving the role of Egypt in regional leadership. SDS represents a roadmap for maximizing competitive advantage to achieve the dreams and aspirations of Egyptians in a dignified and decent life.

It also represents an embodiment of the new constitution's spirit, setting welfare and prosperity as the main economic objectives, to be achieved via sustainable development, social justice, and balanced, geographical and sectoral growth. Therefore, SDS has been developed according to a participatory strategic planning approach; as various civil society representatives, national and international development partners and government agencies have collaborated to set comprehensive objectives for all pillars and sectors of the country.

The current local, regional and global circumstances give the SDS a comparative advantage and importance; for revisiting the strategic vision to cope with international updates and developments. Thus, helping Egypt recover and achieve specific objectives.

SDS has followed the sustainable development principle as a general framework for improving the quality of lives and welfare, taking into consideration the rights of new generations in a prosperous life; thus, dealing with three main dimensions; economic, social, and environmental dimensions.

In addition, SDS is based upon the principles of "inclusive sustainable development" and "balanced regional development", emphasizing the full participation in development, and ensuring its yields to all parties. The strategy, as well, considers equal opportunities for all, closing development gaps, and efficient use of resources to ensure the rights of future generations.

PARTICIPATORY PLANNING APPROACH TO PREPARE THE SUSTAINABLE DEVELOPMENT STRATEGY (SDS): EGYPT VISION 2030

- Many sessions and workshops have been held, with the active participation of experts, academics, private sector representatives, civil society, government officials and international development institutions.
- The SDS team included representatives from all groups including youth, woman, and disabled.
- All development partners participated in the SDS workshops, which witnessed conflicts, different views, and debates. At the end, partners reached a unified vision and agreed on objectives, KPIs, programs and projects.
- SDS team periodically communicated the main results of each stage of the SDS preparation plan with the media. In addition, SDS updates were continuously published on MoP website and social media pages.
- The strategy was presented for community dialogue in many different local and international forums and seminars. The outcomes of working groups have been revised by all relevant ministries and agencies, and the final document included all comments and feedback.
- An electronic portal and social media pages were launched to present the strategy and allow for interaction with all citizens.

By 2030, the new Egypt will achieve a competitive, balanced, diversified and knowledge-based economy, characterized by justice, social integration, and participation, with a balanced and diversified ecosystem, benefiting from its strategic location and human capital to achieve sustainable development for a better life to all Egyptians.

What are EGYPT's 2030 SDGs?

STRATEGIC VISIONS OF SDS PILLARS

THE ECONOMIC DIMENSION

First pillar: Economic development

By 2030,the Egyptian economy is a balanced, knowledge-based, competitive ,diversified, market economy, characterized by a stable macroeconomic environment, capable of achieving sustainable inclusive growth. An active global player responding to international developments, maximizing value added, generating decent and productive jobs, and a real GDP per capita reaching a high-middle income country's level.

Second pillar: Energy

An energy sector meeting national sustainable development requirement and maximizing the efficient use of various traditional and renewable resources contributing to economic growth, competitiveness, achieving social justice, and preserving the environment.A renewable energy and efficient resource management leader, and an innovative sector capable of forecasting and adapting to local, regional and international developments and complying with SDGs.

Third pillar: Knowledge, innovation and scientific research

A creative and innovative society producing science, technology and knowledge, within a comprehensive system ensuring the developmental value of knowledge and innovation using their outputs to face challenges and meet national objectives.

Fourth pillar: Transparency and efficient government institutions

An efficient and effective public administration sector managing State resources with transparency, fairness, and flexibility. Subject to accountability, maximizing citizens, satisfaction and responding to their needs.

THE SOCIAL DIMENSION

Fifth pillar: Social justice

By 2030, Egypt is a fair interdependent society characterized by equal economic, social, political rights and opportunities realizing social inclusion. A society that supports citizens, the right in participation based on efficiency and according to law, encouraging social mobility based on skills. A society that provides protection, and support to marginalized and vulnerable groups.

Sixth pillar: Health

All Egyptians enjoy a healthy, safe, and secure life through an integrated, accessible, high quality, and universal healthcare system capable of improving health conditions through early intervention, and preventive coverage. Ensuring protection for the vulnerable and achieving satisfaction of citizens and health sector employees. This will lead to prosperity, welfare, happiness, as well as social and economic development, which will qualify Egypt to become a leader in the field of healthcare services and research in the Arab world and Africa.

Seventh pillar: Education and training

A high-quality education and training system is available to all, without discrimination within an efficient, just, sustainable, and flexible institutional framework. Providing the necessary skills for students and trainees to think creatively and empower them technically and technologically.

Contributing to the development of a proud, creative, responsible, and competitive citizen who accepts diversity and differences and is proud of his country's history. Eighth pillar: Culture

A system of positive cultural values respecting diversity and differences. Enabling citizens to access knowledge, building their capacity to interact with modern developments, while recognizing their history and cultural heritage. Giving them the wisdom of freedom of choice and of cultural creativity. Adding value to the national economy, representing Egypt's soft power at regional and international levels.

THE ENVIRONMENT DIMENSION

Ninth Pilar: Environment

The environment is integrated in all economic sectors to preserve natural resources and support their efficient use and investment, while ensuring next generations' rights. A clean, safe, and healthy environment leads to diversified production resources and economic activities, supporting competitiveness, providing new jobs, eliminating poverty and achieving social justice.

Tenth pillar: Urban development

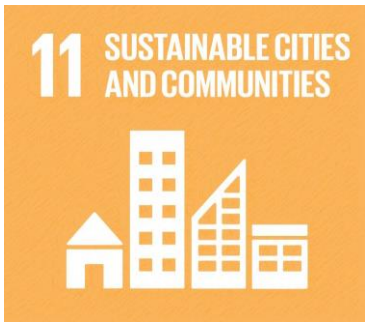
A balanced spatial development management of land and resources to accommodate population and improve the quality of their lives.

What are the main Engineering Applications that will be considered for Projects (2025-2026) aiming at SDGs?

Ensure availability and sustainable management of water and sanitation for all.

Ensure access to affordable, reliable sustainable and modern energy for all.

Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.



Make cities and human settlements inclusive safe, resilient and sustainable

Ensure sustainable consumption and production patterns.

Take urgent action to combat climate change and its impacts.

Conserve and sustainably use the oceans, seas and marine resources for sustainable development.

Protect, restore, and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, and halt and reverse land degradation and halt biodiversity loss.

Implementation of Egypt's 2030 Sustainable Development Goals (SDGs) through AIET Projects

This graduation project is aligned with Egypt Vision 2030 and the Sustainable Development Goals by demonstrating how digital engineering solutions can support healthcare, infrastructure, innovation, and safer communities. The software part of the project focuses on a smart hospital case study where network

Expected SDGs Achieved by the Project

- Good Health and Well-Being: The ClinicEase system improves access to healthcare booking, patient appointment tracking, and medical scan access.
- Industry, Innovation and Infrastructure: The project demonstrates resilient digital infrastructure through Packet Tracer network design, cloud architecture, and modular software implementation.
- Sustainable Cities and Communities: A smart hospital environment improves service organization and supports safer, more efficient healthcare facilities.
- Quality Education: The project applies practical engineering knowledge in networking, cloud computing, web development, mobile development, and system integration.

Mission of Electronics and Communication Engineering Program

Vision:

The department seeks to graduate scientifically distinguished engineers who have the ability to participate in the fields of electronics and communications engineering and scientific research based on high-quality education compatible with regional and international standards and develop their professional capabilities for community service and environmental development.

رؤية قسم الإلكترونيات والاتصالات:

يسعى القسم إلى تخريج مهندسين متميزين علمياً لهم القدرة على المشاركة في مجالات هندسة الإلكترونيات والاتصالات والبحث العلمي اعتماداً على تعليم ذات جودة عالية متوافقة مع المعايير الإقليمية والدولية وتنمية قدراتهم المهنية لخدمة المجتمع وتنمية البيئة.

Mission:

The department aims to graduate generations of engineers in the field of electronics and communications engineering who have the spirit of innovation, technological creativity and the ability to compete in the labor market through the application of the latest teaching and learning systems that keep pace with scientific and research progress in line with the developments of the times to achieve sustainable development and contribute to community service and environmental development.

رسالة قسم الإلكترونيات والاتصالات:

يهدف القسم إلى تخريج أجيال من المهندسين في مجال هندسة الإلكترونيات والاتصالات لديهم روح الابتكار والإبداع التكنولوجي والقدرة على المنافسة في سوق العمل من خلال تطبيق أحدث نظم التعليم والتعلم والتي تواكب التقدم العلمي والبحثي بما يتماشى مع مستجدات العصر لتحقيق التنمية المستدامة والمساهمة في خدمة المجتمع وتنمية البيئة.

Declaration

This report has been prepared based on our own project. Where other published and unpublished source materials have been used, these have been acknowledged.

We, the students of the project entitled Integrated Software Engineering Solutions, declare that the work presented in this report has been prepared according to academic integrity rules and intellectual property regulations.

Student Name and Signature:

No.	Student Name	Signature
1	Ali Ayman Mohamed Gamal Eldeen	.....
2	Mohamed Elshazly AbdElaziz Mobarak	.....
3	Kyrellos Hany Saied Fahem	.....
4	Marwan Samy Mostafa Elzwawy	.....
5	Rewan Alaa Abd Elmaksoud	.....
6	Eman Ahmed Ghaith	.....
7	Alaa Reda Mohammad Abo Mehana	.....
8	Rowaida Ahmed Abd El-Wahab	.....
9	Aya Ahmed Khattab	.....
10	Mohamed Ahmed Mohamed Aly	.....

Date of Submission: 22\6\2026

Acknowledgements

We would like to express our sincere appreciation to our supervisors, Dr. Ola and Dr. Amr, for their continuous guidance, support, and valuable feedback throughout the development of this graduation project. Their academic direction helped us organize the project, improve its technical quality, and present it in a professional engineering form.

We would also like to thank the lecturers, teaching assistants, and staff members of Alexandria Higher Institute of Engineering & Technology for their support during our study period and during the preparation of this project. Their effort contributed to building the technical knowledge required to complete the network, web, mobile, and cloud parts of the project.

Finally, we would like to thank our families and colleagues for their encouragement, patience, and support during the different stages of research, implementation, testing, and documentation.

Table of contents

Abstract

This report presents the software part of the graduation project entitled Integrated Software Engineering Solutions. The wider project includes both software and hardware tracks; however, this book focuses only on the software engineering services developed for the ClinicEase smart healthcare case study. The project addresses the need for hospitals to use connected digital solutions instead of isolated systems.

The proposed prototype combines three main software service categories. The first category is the hospital network design, implemented and simulated using Cisco Packet Tracer. It includes hierarchical topology design, VLAN segmentation, DHCP services, wireless connectivity, CCTV and IoT monitoring, redundancy using HSRP and STP, and security features such as SSH, port security, and DHCP snooping. This provides a reliable infrastructure foundation for the smart hospital environment.

The second category covers the web system, UI/UX design, and mobile application. These interfaces support guests, patients, and administrators through role-based workflows. Patients can browse specialists, select doctors, book appointments, view appointment history, and access medical scans. Administrators can monitor appointments, update appointment status, manage patient records, and upload scan files.

The third category is the cloud and backend solution. AWS API Gateway, Lambda, DynamoDB, and S3 are used to support serverless backend functions for authentication, appointment management, medical scan upload and retrieval, and future hardware-cloud integration. Overall, the project demonstrates a complete software engineering workflow from requirements and design to implementation, testing, integration, documentation, and future improvement. The final result is a realistic smart hospital software prototype that can be expanded into a more secure, scalable, and production-ready healthcare platform.

Project Specification

Project Identification

Item	Description
Project Title	Integrated Software Engineering Solutions
Program	Electronics and Communication Engineering (ECE)
Academic Year	2025–2026
Supervisors	Dr. Ola and Dr. Amr
Project Type	Software engineering and smart healthcare prototype
Case Study	Smart hospital / ClinicEase healthcare management system
Main Software Services	Network design, web development, mobile application development, cloud backend solutions, and system integration

Project Need

Modern organizations, especially hospitals, need digital systems that are not isolated from each other. A hospital may require a secure network, a public website, patient-facing applications, an administrative dashboard, cloud storage, backend APIs, and secure access to medical files. In many real environments, these services are delivered by separate vendors, which can increase cost, reduce consistency, and make integration difficult. This project addresses that need by presenting one integrated software engineering solution that combines several services under a unified technical concept.

Project Objectives

- Design a reliable and secure hospital network infrastructure using Cisco Packet Tracer.
- Develop a healthcare web system that supports guest, patient, and admin flows.
- Develop a mobile application for remote appointment booking and patient interaction.
- Design and implement cloud backend services for authentication, appointments, and medical scans.
- Demonstrate how web, mobile, network, and cloud components can be integrated into one software solution.
- Document the project professionally according to the AIET final year project report structure.

Expected Features

- Role-based user experience for guests, patients, and administrators.
- Appointment booking and appointment history features.
- Admin dashboard with appointment status management.
- Medical scan upload and patient scan retrieval workflow.
- Cloud data storage using DynamoDB tables and S3 files.
- Packet Tracer hospital network with VLANs, redundancy, security, IoT devices, and wireless access.
- Mobile application with Flutter interface, BLoC/Cubit architecture, localization, and notifications.

Constraints and Assumptions

The project is implemented as an academic prototype. Some parts, such as frontend authentication and localStorage-based simulation in early website versions, are designed for demonstration and were later mapped to cloud architecture. The network implementation is simulated using Cisco Packet Tracer, which may not support every real-world feature with complete production accuracy. The cloud backend is based on serverless AWS services suitable for prototype deployment and future scalability. The project does not include the full hardware implementation in this software report, although it is designed to be compatible with future hardware-cloud integration.

Tools and Technologies

Area	Tools and Technologies
Network Design	Cisco Packet Tracer, VLANs, HSRP, STP, DHCP, SSH, Port Security, DHCP Snooping, IoT Server
Web Development	HTML, CSS, Bootstrap, JavaScript, localStorage prototype, cloud-ready frontend logic
Mobile Development	Flutter, Dart, BLoC/Cubit, ScreenUtil, L10n, Lottie Animations, local notifications
Cloud Backend	AWS S3, AWS Lambda, API Gateway, DynamoDB, S3 Presigned URLs
Documentation	AIET report template, technical reports, screenshots, diagrams, and validation evidence

Chapter 1: Introduction

1.1 Project Background

Digital transformation has become an essential requirement in healthcare environments. Hospitals depend on accurate communication, secure information storage, fast appointment handling, and reliable access to patient records. A hospital is not only a building with medical devices; it is also a connected technical environment where networks, software systems, mobile applications, and cloud services must work together. Therefore, a successful smart hospital system requires integrated engineering solutions rather than separate disconnected components.

The project entitled Integrated Software Engineering Solutions presents a prototype of a professional technical team that provides several software and infrastructure services to clients. Instead of building only one standalone website or one isolated network, the project demonstrates a complete service model. The software team can design the network infrastructure, build web and mobile interfaces, create UI/UX flows, develop backend logic, and connect the system to cloud services.

A smart hospital environment was selected as the case study because hospitals clearly need secure networking, patient-facing applications, administrative tools, and cloud-based storage. The ClinicEase healthcare prototype represents the software implementation of this concept. Patients can browse doctors, book appointments, and access scans, while administrators can monitor appointments and upload medical scan records. In parallel, the hospital network design provides the communication foundation required for departments, servers, cameras, wireless users, and IoT devices.

1.2 Problem Statement

Many small and medium healthcare facilities face difficulty when trying to move from traditional manual workflows to digital workflows. Appointment booking may depend on physical presence or phone calls, patient records may be scattered, and management dashboards may be unavailable. At the infrastructure level,

networks may be poorly segmented, insecure, or unable to support critical hospital services. When every technical part is implemented separately, the final system may suffer from weak integration, repeated data entry, poor reliability, and limited scalability.

This project solves the problem by presenting an integrated software engineering approach. The hospital case study is designed to show how one team can deliver network design, web interfaces, mobile interfaces, cloud backend, and integration planning in one consistent architecture. The final prototype is not only a user interface; it is a complete engineering story that connects requirements, design, implementation, testing, and future improvement.

1.3 Project Objectives

- To create a realistic software engineering solution for a smart hospital case study.
- To design a secure and scalable hospital network that supports departments, servers, cameras, wireless devices, and IoT components.
- To implement a web-based healthcare prototype that supports guest browsing, patient booking, and admin dashboard operations.
- To implement a mobile application that provides a simple and responsive patient booking experience.
- To connect major workflows to cloud backend services using serverless AWS architecture.
- To document the architecture, workflows, technologies, results, challenges, and future work in a professional academic report.

1.4 Project Scope

This report covers the software part only of the wider graduation project. The complete project includes both software and hardware tracks; however, this book is limited to network design, web development, UI/UX design, mobile application development, cloud backend, and system integration. The hardware phase is treated as a separate part and is not documented in detail in this report. Nevertheless, the cloud solution is prepared

to support future hardware–cloud connection, so sensor data, automation systems, and physical devices can later be integrated with the same general software architecture.

1.5 Project Outline

The report is organized into eight chapters. Chapter 1 introduces the project background, problem, objectives, and scope. Chapter 2 explains the integrated software engineering concept and the hospital case study. Chapter 3 presents the Cisco Packet Tracer network design solution. Chapter 4 documents the web system and UI/UX solution. Chapter 5 explains the mobile application. Chapter 6 presents the AWS cloud and backend solution. Chapter 7 discusses integration, results, and implementation challenges. Chapter 8 concludes the report and proposes future work.

Chapter 2: Integrated Software Engineering Solutions Overview

2.1 Team Concept

The project is based on the idea of a specialized software engineering team that provides several digital services through one organized technical workflow. The team model is important because real clients rarely need only one service. A client may need a website, a mobile application, a network design, secure data storage, and cloud deployment at the same time. The proposed team can handle these requirements as an integrated solution instead of delivering isolated pieces.

The team roles include network design, cloud solution implementation, web development, mobile application development, documentation, and project leadership. This role distribution allows every part to be developed by a responsible subgroup while still contributing to one final integrated system.

2.2 Software Services Provided

Service	Description	Project Evidence
Network Design	Designing secure, segmented, redundant, and scalable hospital network infrastructure.	Cisco Packet Tracer topology, VLANs, HSRP, STP, DHCP, SSH, cameras and IoT.
Web Development and UI/UX	Building the healthcare web interface and user flows for guests, patients, and administrators.	HTML, CSS, Bootstrap, JavaScript, dashboard pages, booking pages, scan pages.
Mobile Application Development	Building a cross-platform mobile app for patient booking and notifications.	Flutter, Dart, BLoC/Cubit, localization, appointment screens, notifications.
Cloud and Backend Solutions	Moving major workflows from local prototype behavior into cloud-connected backend services.	AWS S3, Lambda, API Gateway, DynamoDB, presigned URLs.
System Integration	Connecting services into a complete smart hospital workflow.	Shared case study, role-based flows, appointment data, scan data, cloud APIs.

2.3 Hospital Case Study

The hospital environment was selected because it is a practical example of an organization that needs both software and infrastructure. The case study includes public users who browse services, patients who book appointments, administrators who manage appointments and scans, and network infrastructure that connects

hospital departments and devices. This makes the case study suitable for proving the value of integrated software engineering solutions.

2.4 System Users and Roles

User Type	Main Responsibilities and Access
Guest	Browses the public website, explores specialists, reads service information, and moves to sign in or sign up when protected actions are needed.
Patient	Creates account, signs in, selects specialists and doctors, books appointments, views upcoming and past appointments, and accesses medical scans.
Admin	Signs in to the dashboard, views system metrics, manages appointment status, views patient records, and uploads medical scans.
Network Administrator	Manages the simulated hospital network, VLANs, IP addressing, access switches, security, and troubleshooting.
Cloud Backend Operator	Manages API Gateway, Lambda functions, DynamoDB tables, S3 storage, and backend testing.

2.5 Overall System Architecture

The overall architecture contains four main layers. The infrastructure layer is represented by the Packet Tracer hospital network, which includes routers, switches, VLANs, servers, cameras, wireless users, and IoT elements. The presentation layer includes the website and the mobile application. The backend layer includes API Gateway and Lambda functions that receive requests and execute business logic. The data and file storage layer includes DynamoDB tables for structured records and S3 buckets for website hosting and medical scan files.

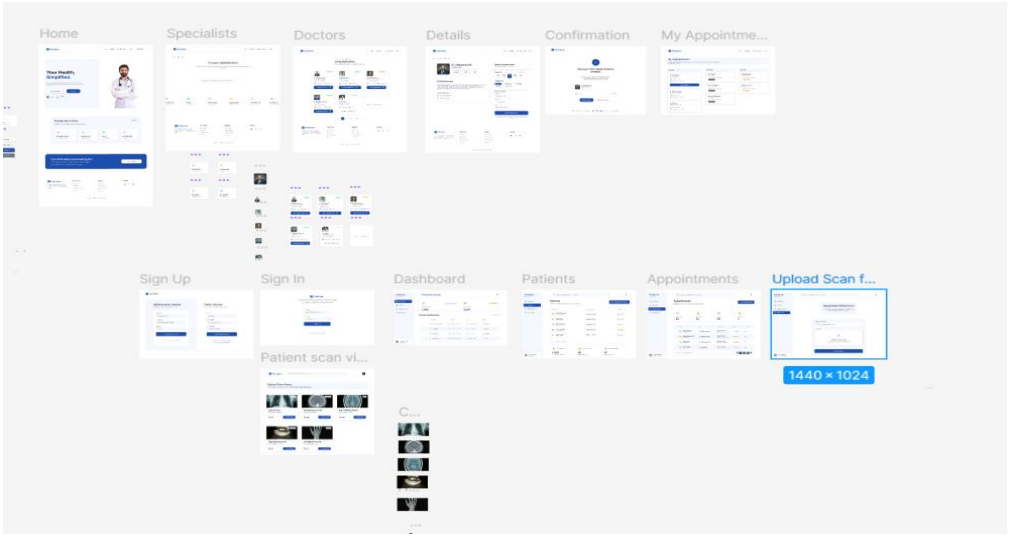


Figure 2.1. Overall web system and architecture visual evidence from the developed documentation.

Chapter 3: Network Design Solution

This chapter presents the complete hospital network design solution implemented using Cisco Packet Tracer. The section has been expanded to include the full design objective, topology model, VLAN schema, redundancy protocols, security configuration, DHCP and wireless setup, CCTV/IoT integration, troubleshooting evidence, and the complete CLI commands used during implementation.

Expanded Chapter Contents

3.1 Project Introduction and Requirements

Objective: The objective of this project is to design and implement a modern, secure, and scalable network infrastructure for a hospital. The design primarily focuses on ensuring High Availability and having No Single Point of Failure in the network core, all while implementing secure and logical network segmentation.

Overall Architecture: The design is based on the Hierarchical Network Design model, which divides the network into distinct layers:

Core Layer: Represents the "brain" and backbone of the network, responsible for high-speed routing. (Device:R1, R2, and CORE_SW)

Distribution Layer: Acts as the mediator that connects the Core Layer to the Access Layer and provides redundancy. (Devices: SW1 and SW2)

Access Layer: The layer where end-users and devices (PCs, phones, etc., in the departments) connect to the network. (The 5 switches to be added later).

The first step:

- We used the model to simulate the exact design.

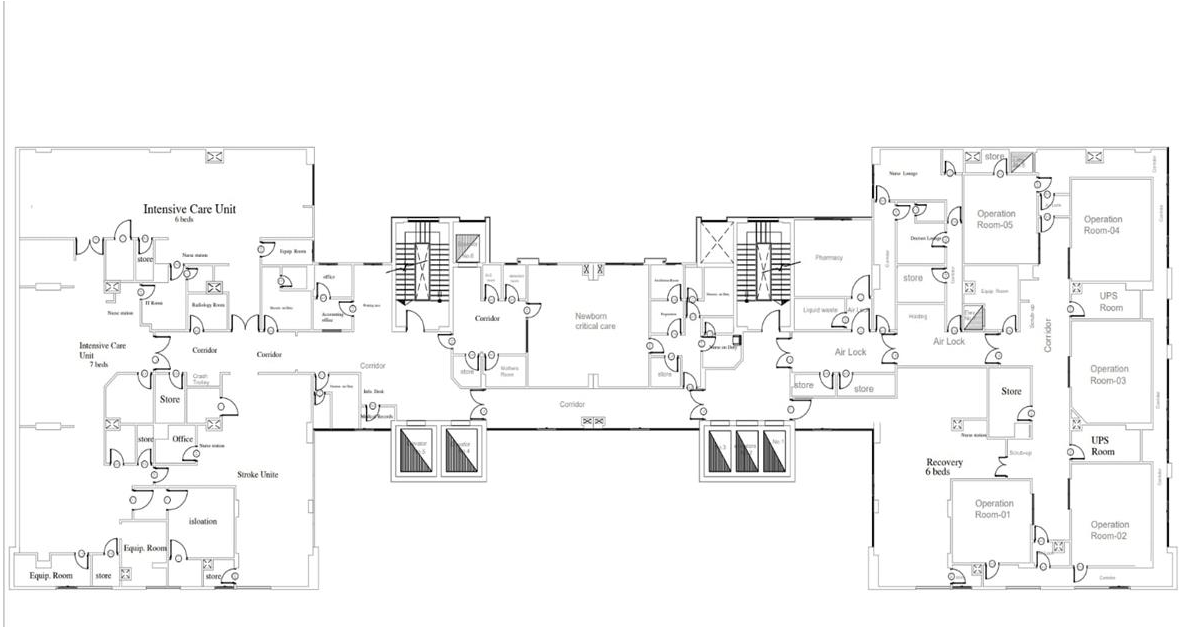


Figure 3.1. Network design evidence extracted from the Packet Tracer implementation report.

The second step:

- we replicated the model's layout precisely in Packet Tracer for simulation.
- We defined placement, color, and equipment for each room.

Figure 3.2. Network design evidence extracted from the Packet Tracer implementation report.

The third step:

- determine the required equipment for each room.

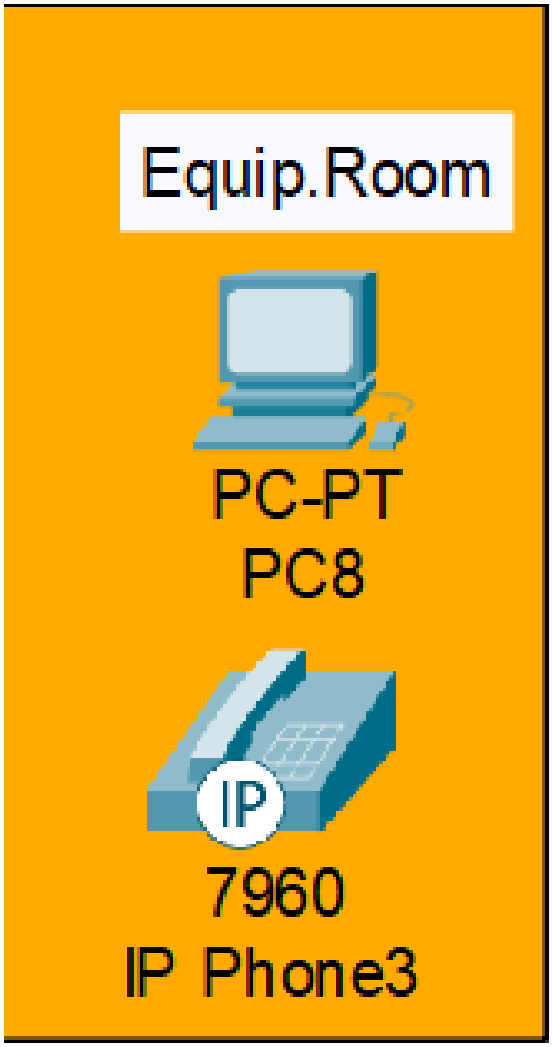


Figure 3.3. Network design evidence extracted from the Packet Tracer implementation report.

Figure 3.4. Network design evidence extracted from the Packet Tracer implementation report.

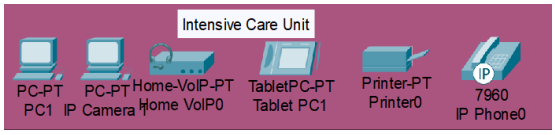


Figure 3.5. Network design evidence extracted from the Packet Tracer implementation report.

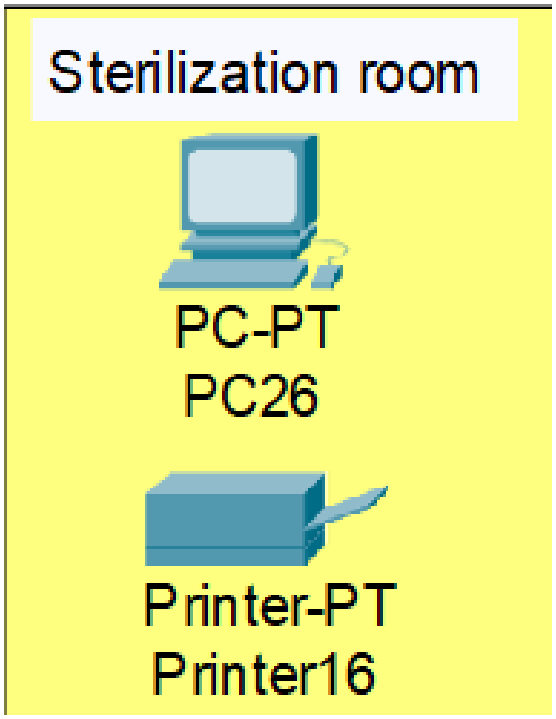


Figure 3.6. Network design evidence extracted from the Packet Tracer implementation report.

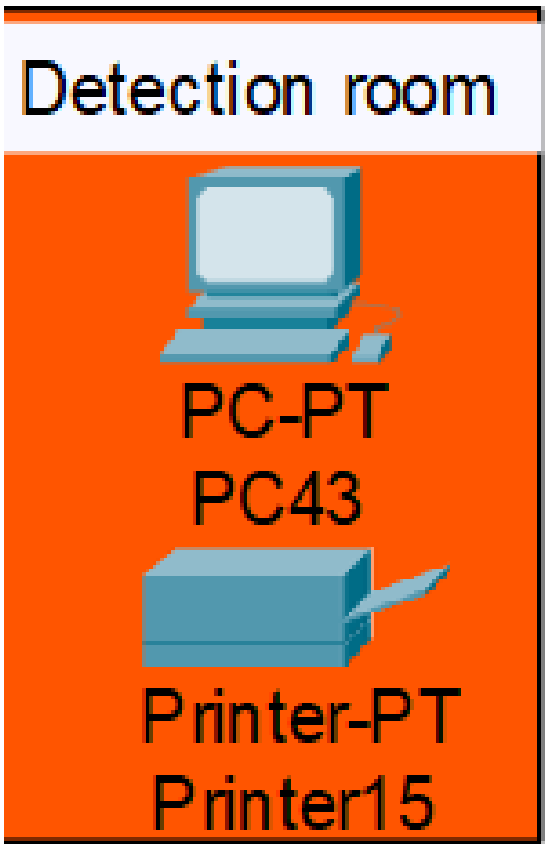


Figure 3.7. Network design evidence extracted from the Packet Tracer implementation report.

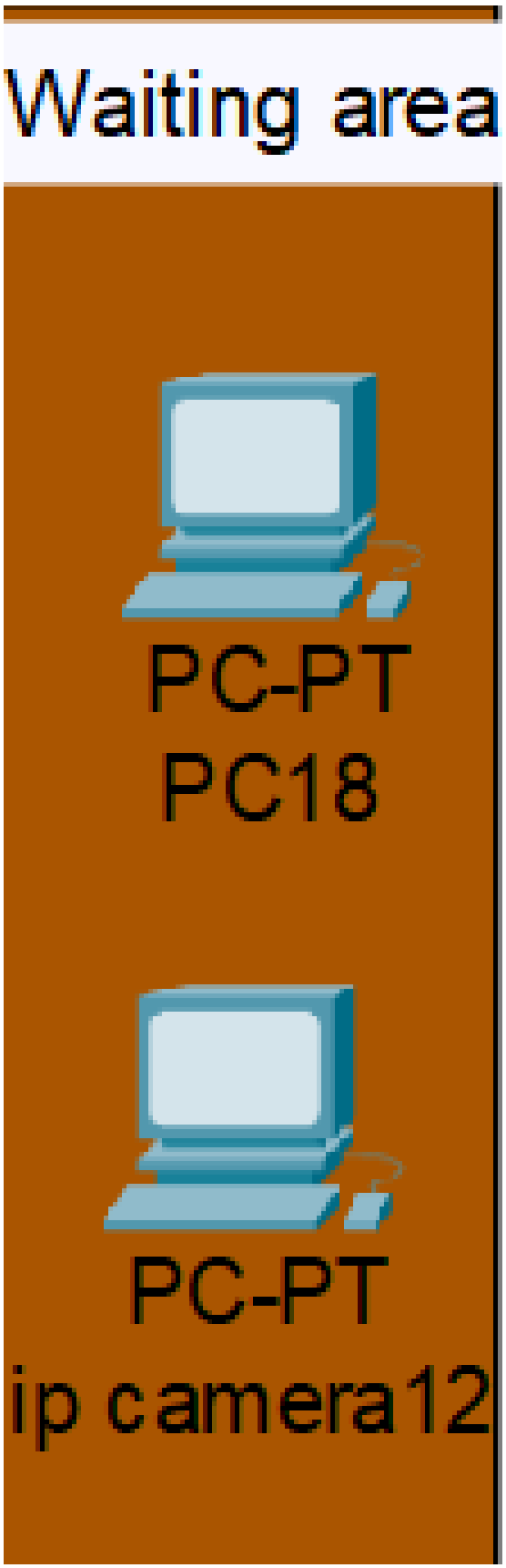


Figure 3.8. Network design evidence extracted from the Packet Tracer implementation report.

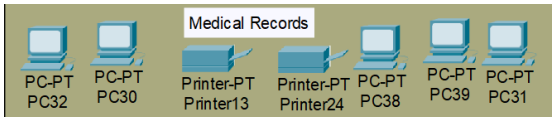


Figure 3.9. Network design evidence extracted from the Packet Tracer implementation report.

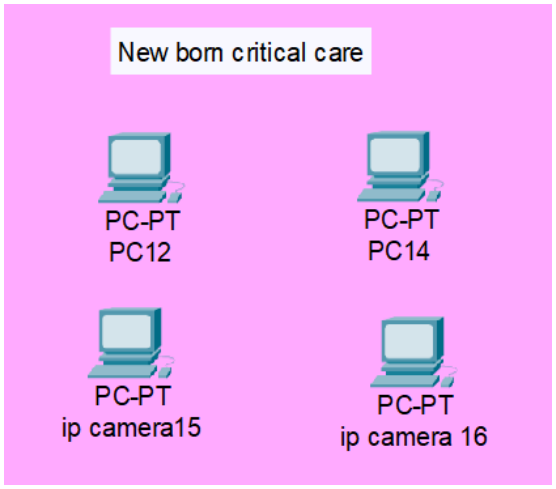


Figure 3.10. Network design evidence extracted from the Packet Tracer implementation report.

Figure 3.11. Network design evidence extracted from the Packet Tracer implementation report.

Figure 3.12. Network design evidence extracted from the Packet Tracer implementation report.

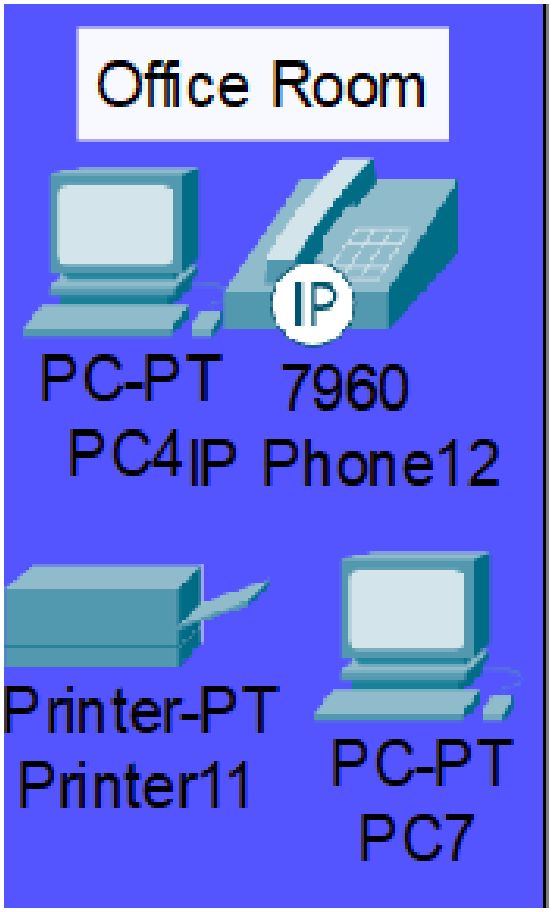


Figure 3.13. Network design evidence extracted from the Packet Tracer implementation report.

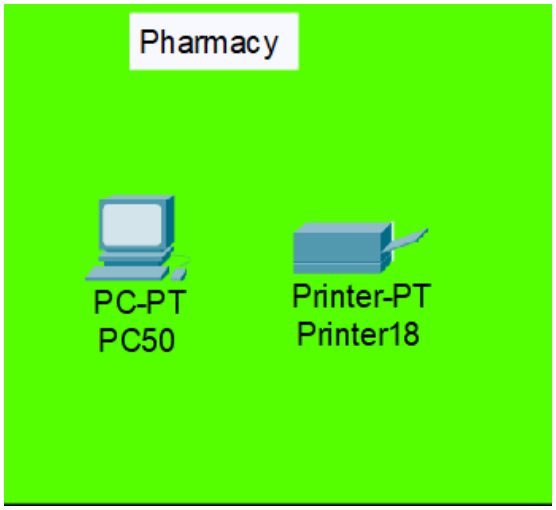


Figure 3.14. Network design evidence extracted from the Packet Tracer implementation report.

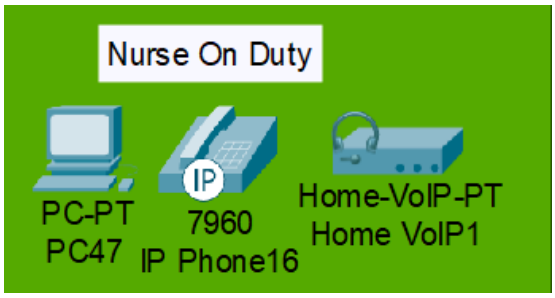


Figure 3.15. Network design evidence extracted from the Packet Tracer implementation report.

- The number of devices we used :

Device Type	Quantity Used
Switch	8
Router	2
PC	80
Printer	26
Server	1
Ip phone	33
Tablet	6
Camera	5
Laptop	8
Access Point	5
Motion Detector	5

The fourth step:

Adopting a Hierarchical Network Topology position the IT room at the top of the design.

3.2 Network Design and Topology

3.2.1 Logical Topology

The design relies on a "triangle" of switches in the network core to ensure redundancy, along with two routers to ensure constant internet connectivity.

Routers (R1, R2): They use the HSRP protocol to provide a single virtual gateway. If one router fails, the other takes over its role instantly.

Core Switch (CORE_SW): This device acts as the L3 Router for the internal network (Inter-VLAN Routing). It is the central aggregation point for all subnets (VLANs).

Distribution Switches (sw1, sw2): These act as L2 switches, providing redundant physical paths to the Core Switch. The STP protocol manages these paths to prevent network loops.

3.2.2 Network Diagram

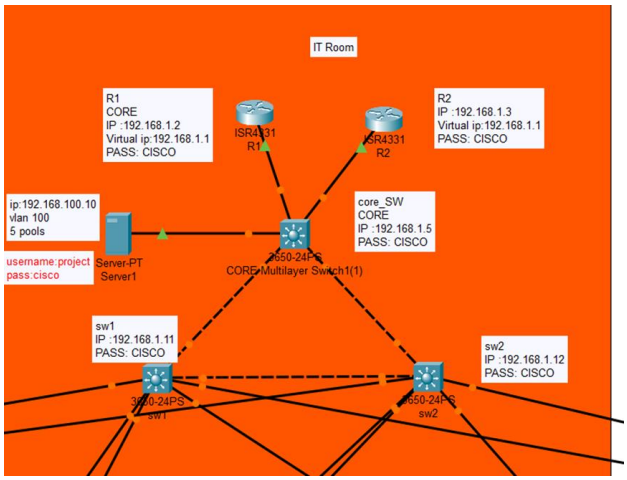


Figure 3.16. Network design evidence extracted from the Packet Tracer implementation report.

3.2.3 VLAN Schema

The network is segmented into 5 primary VLANs for different departments, in addition to a management VLAN.

VLAN ID	Name	Gateway (on CORE_SW)	Network
1	default (Management)	192.168.1.5	192.168.1.0 /24
10	VLAN_10	192.168.10.1	192.168.10.0 /24
20	VLAN_20	192.168.20.1	192.168.20.0 /24
30	VLAN_30	192.168.30.1	192.168.30.0 /24
40	VLAN_40	192.168.40.1	192.168.40.0 /24
50	VLAN_50	192.168.50.1	192.168.50.0 /24

Table 3.1. VLAN schema and gateway addressing plan.

3.3 Protocols and Technologies Used

3.3.1 HSRP (Hot Standby Router Protocol)

Function: Provides gateway redundancy (High Availability) for the network's internet connection.

How it works: We created an HSRP group between R1 and R2. They share a virtual IP (192.168.1.1). R1 is configured as the Active router (priority 105) and R2 is the Standby (priority 100). All devices on the network use this virtual IP as their default gateway. If R1 fails, R2 immediately takes over the Active role.

3.3.2 Inter-VLAN Routing

Function: Allows different VLANs to communicate with each other (e.g., allowing the Pharmacy Vlan10 to communicate with Reception Vlan20).

How it works: Instead of using a separate router, we enabled the ip routing feature on the CORE_SW (a Multilayer Switch). We then created Switched Virtual Interfaces (SVIs) for each VLAN (e.g., interface Vlan10) and assigned them an IP address, turning them into gateways for their respective VLANs.

3.3.3 Spanning Tree Protocol (STP)

Function: Prevents network "loops" in a redundant topology.

How it works: Because we created a physical "triangle" of connections between CORE_SW, sw1, and sw2 for redundancy, STP automatically activates. It calculates the best path and logically blocks the redundant paths to prevent a loop. If the primary path fails, STP unblocks the redundant path within seconds, ensuring network uptime.

3.4 IT Room and Core Device Configuration

3.4 Device Configurations and Full CLI Commands

This section details the function and basic configuration for each core device.

3.4.1 Router R1 (Primary)

Function: The primary (Active) router for internet egress. It holds the highest HSRP priority to ensure it is the default choice.

Important CLI commands are listed in Appendix D.

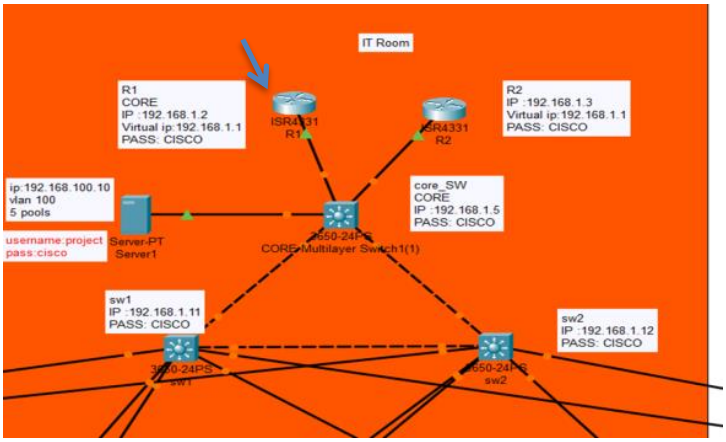


Figure 3.17. Network design evidence extracted from the Packet Tracer implementation report.

3.4.2 Router R2 (Standby)

Function: The standby (Backup) router. It monitors R1 and is ready to take over the Active role instantly if R1 fails.

Important CLI commands are listed in Appendix D.

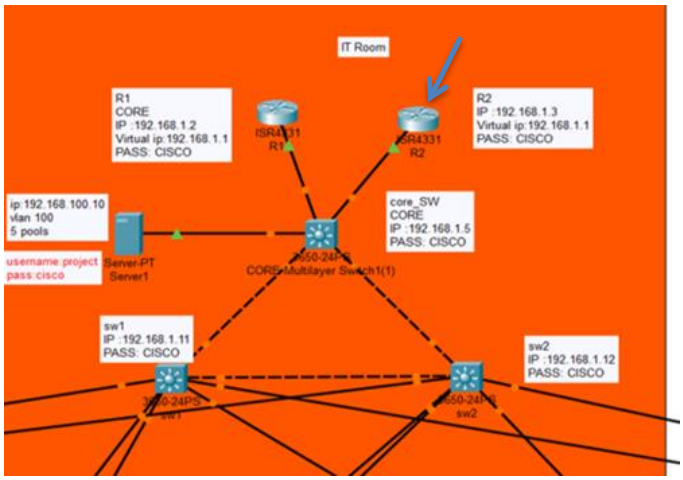


Figure 3.18. Network design evidence extracted from the Packet Tracer implementation report.

3.4.3 Switch CORE_SW (Core)

Function: The "brain" of the internal network. It performs all L3 routing between VLANs, serves as the gateway for all departments, and routes internet-bound traffic to the HSRP virtual IP.

Important CLI commands are listed in Appendix D.

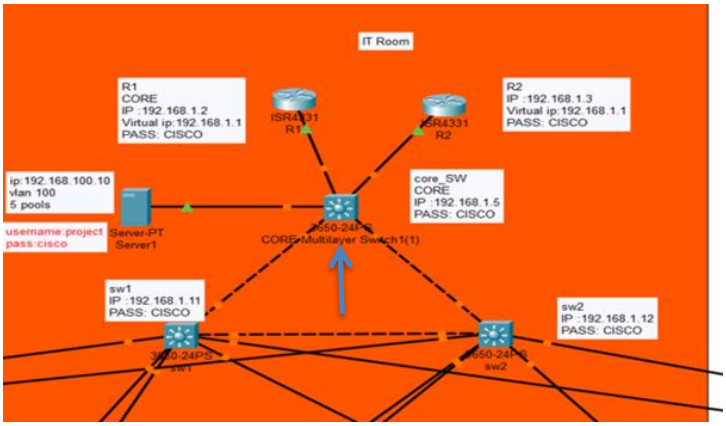


Figure 3.19. Network design evidence extracted from the Packet Tracer implementation report.

3.4.4 Distribution Switches SW1 and SW2

Function: L2 switches that aggregate connections from the Access layer and provide redundant paths to the Core. (Configurations are nearly identical for both).

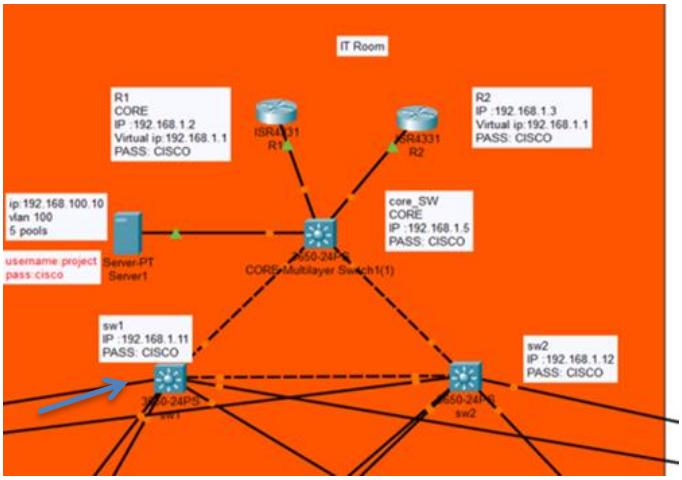


Figure 3.20. Network design evidence extracted from the Packet Tracer implementation report.

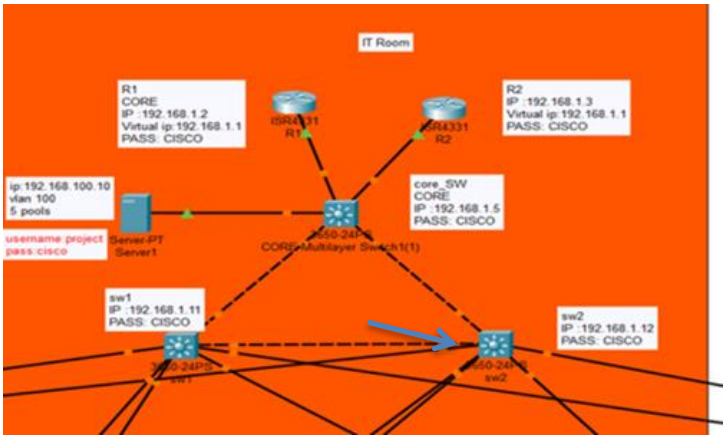


Figure 3.21. Network design evidence extracted from the Packet Tracer implementation report.

3.4.5 DHCP Server

Function: A central server to automatically distribute IP addresses

Configuration (GUI-based, not commands)

IP Configuration (Desktop)

DHCP (Services)

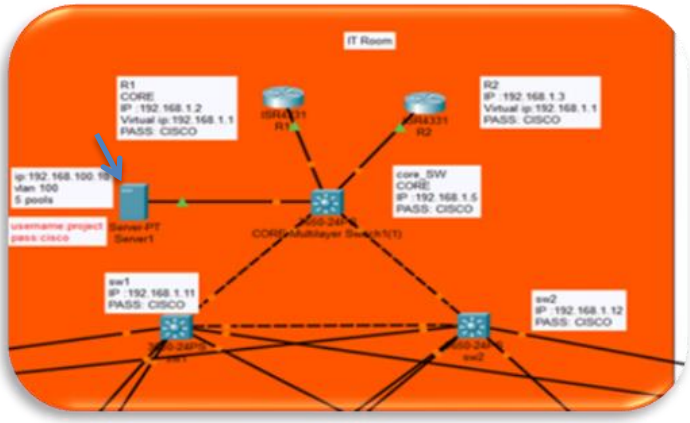


Figure 3.22. Network design evidence extracted from the Packet Tracer implementation report.

3.5.1 IT Room Configuration Conclusion

In the IT room, the core switch was configured as a Layer 3 switch using IP routing. Multiple VLANs were created, and each VLAN was assigned an IP address to act as its gateway. Inter-VLAN routing was enabled, trunk ports were configured, and IP helper-address was used to connect VLANs to the DHCP server. Finally, a default route was added and the device was secured with passwords.

3.6 VLAN Implementation and Network Segmentation

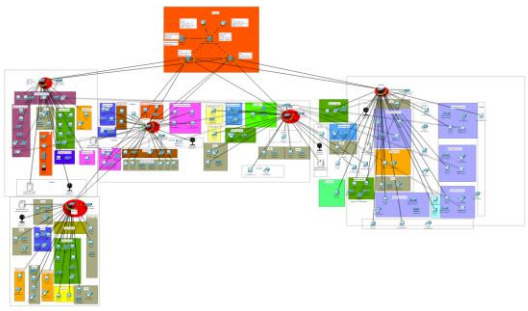


Figure 3.23. Network design evidence extracted from the Packet Tracer implementation report.

VLANs (Virtual Local Area Networks) are a technology used to divide a single physical network into multiple, isolated logical networks. In this project, the network was not merely a collection of connected devices; instead, each department (such as the Pharmacy, Operating Rooms, and Administration) was separated into its own Broadcast Domain.

1. Technical Significance

Traffic Isolation: Patient data in the Operating Rooms must remain isolated from administrative staff or visitor traffic to ensure privacy and efficiency.

Security: By restricting communication between VLANs, the network ensures that any security breach or malware in one department cannot spread to the rest of the infrastructure.

Performance Optimization: This design reduces network congestion by limiting broadcast messages (e.g., from the Radiology department) to their specific VLAN, preventing them from flooding the entire hospital network.

2. Functional Departmental Analysis

To ensure efficient network management, enhanced security, and reduced broadcast traffic, the hospital network was logically segmented into five separate VLANs. Each VLAN was assigned to a specific department or functional area within the hospital, allowing better traffic isolation, improved performance, and simplified administration.

3. Connectivity and Backbone Architecture

While VLANs are logically isolated, they are interconnected via central Layer 3 Switches or Routers.

Purpose of Connectivity: To enable devices to send reports and data to the central servers located in the upper tier (the orange section in project).

Trunk Links: The black lines connecting the sub-switches to the core represent Trunk Links. These links carry traffic from multiple VLANs simultaneously using the 802.1Q protocol while maintaining data isolation.

4. Inter-VLAN Routing Control

The communication between VLANs has been restricted using Access Control Lists (ACLs) or by disabling Inter-VLAN Routing.

Security Benefit: This implements a "Zero Trust Security" model. For instance, staff at the reception desk do not require access to data from specialized operating room equipment.

The important VLAN CLI commands are listed in Appendix D.

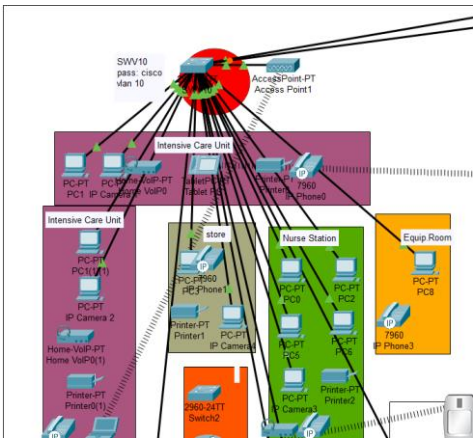


Figure 3.24. Network design evidence extracted from the Packet Tracer implementation report.

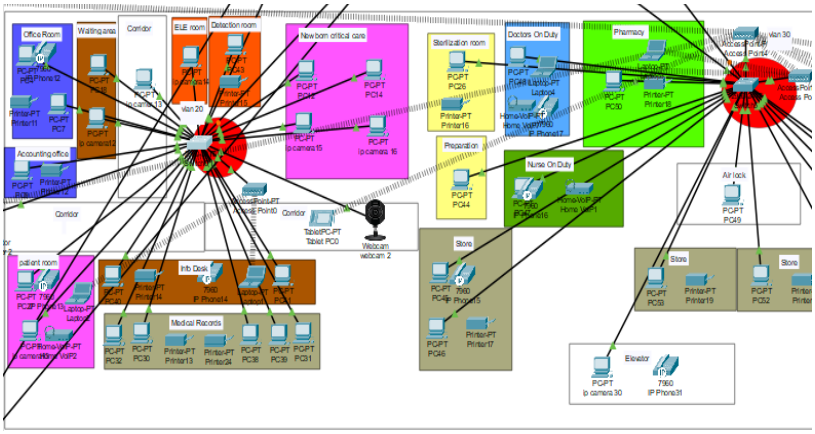


Figure 3.25. Network design evidence extracted from the Packet Tracer implementation report.

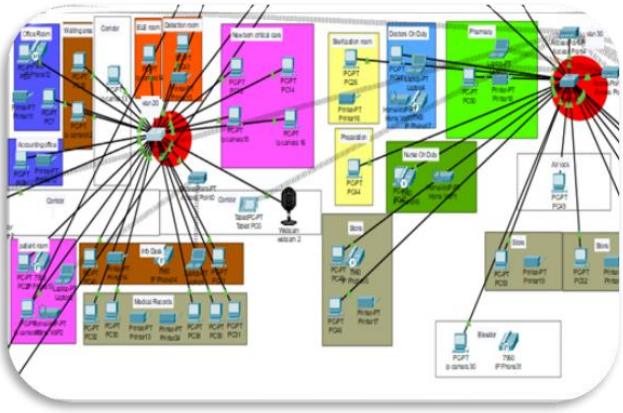


Figure 3.26. Network design evidence extracted from the Packet Tracer implementation report.

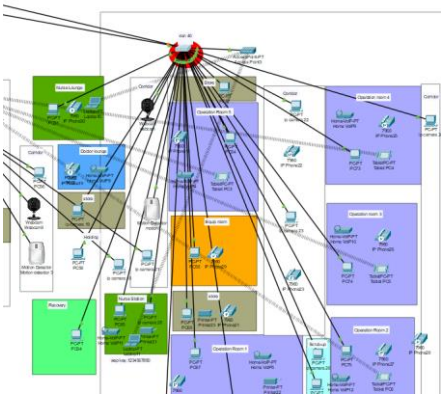


Figure 3.27. Network design evidence extracted from the Packet Tracer implementation report.

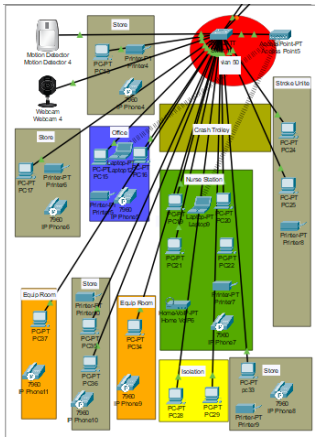


Figure 3.28. Network design evidence extracted from the Packet Tracer implementation report.

3.7 CCTV and IoT Monitoring Integration

The Closed-Circuit Television (CCTV) system has been integrated into the hospital network design to ensure high levels of security and continuous monitoring. The significance of the cameras lies in the following:

Enhancing Hospital Security:

Monitoring entrances, exits, and critical areas (such as server rooms and the pharmacy) to prevent theft or unauthorized access.

Protecting Patients and Medical Staff:

Ensuring patient safety, especially in emergencies, and monitoring visitor behavior within the facility.

Real-time Monitoring:

Enabling hospital management to observe live events directly through the server.

Event Recording:

Retaining video footage for future reference in case of incidents or investigations.

Integration with Other Security Systems:

Connecting with devices like Motion Detectors to identify unusual activity and trigger alerts.

3.7.1 Camera System Components

The following elements were utilized within Cisco Packet Tracer:

Component	Description
Camera (Webcam)	The visual monitoring device.
Motion Detector	Sensors to detect movement and trigger actions.
Switch	The central node for connecting devices within the Local Area Network (LAN).
Server	Used for data storage, management, and hosting the monitoring service.
PCs	Monitoring stations used by security or administration.
Copper Straight-Through Cables	The physical medium used to connect the devices to the switch.

Table 3.2. CCTV and IoT monitoring system components.

3.7.2 Camera Configuration Steps in Packet Tracer

1- Adding Devices

Open the program and select the following:

End Devices →Webcam

Sensors →Motion Detector

Network Devices →Switch (e.g., 2960 Series)

End Devices →Server

2- Connecting Devices

Connect the Webcam and the Motion Detector to the Switch using a Copper Straight-Through Cable.

Connect the Server to the Switch.

Verification: Ensure all devices are connected to the same network (assigned to the same subnet).

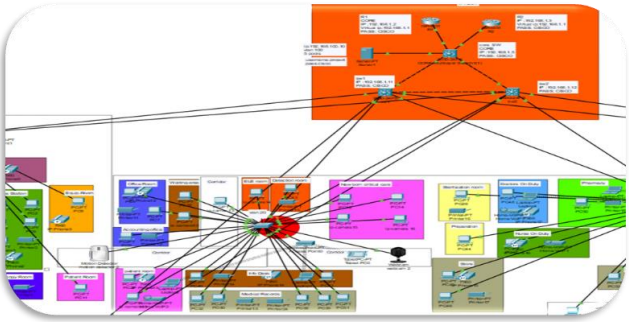


Figure 3.29. Network design evidence extracted from the Packet Tracer implementation report.

3.7.3 IP Addressing Setup for Cameras and Motion Detectors

Device Addressing Table:

Device	IP Address	Subnet Mask
Server	192.168.100.10	255.255.255.0
Camera 1	192.168.10.10	255.255.255.0
Camera 2	192.168.20.19	255.255.255.0
Camera 3	192.168.30.12	255.255.255.0
Camera 4	192.168.40.10	255.255.255.0
Camera 5	192.168.50.12	255.255.255.0
Motion Detector 1	192.168.10.11	255.255.255.0
Motion Detector 2	192.168.20.16	255.255.255.0
Motion Detector3	192.168.30.18	255.255.255.0
Motion Detector 4	192.168.40.12	255.255.255.0
Motion Detector 5	192.168.50.11	255.255.255.0

Table 3.3. Camera and motion detector IP addressing plan.

IP Configuration Steps

The following configuration steps were performed for each device (Camera, Motion Detector, and Server):

Access the Device: Click on the device icon within the Cisco Packet Tracer workspace.

Navigate to IP Settings: Go to the Desktop tab and select the IP Configuration tool.

Enter Addressing Details:

Figure 3.30. Network design evidence extracted from the Packet Tracer implementation report.

3.7.4 IoT Server Integration and Rule-Based Activation

After assigning IP addresses to all devices, the following steps were performed to activate and integrate the IoT system, ensuring a secure connection between the Cameras, Motion Detectors, and the Server.

4.1 Enabling IoT Services on the Server

Open the Server configuration in Cisco Packet Tracer.

Navigate to the Services tab →IoT.

Enable the service by setting the status to ON.

Created a Username & Password to secure the connection between the devices and the server.

4.2 Accessing the Server via Web Browser

From a connected PC, navigate to Desktop →Web Browser.

Enter the Server's IP Address in the URL bar.

Once the login page appeared, entered the:

Username

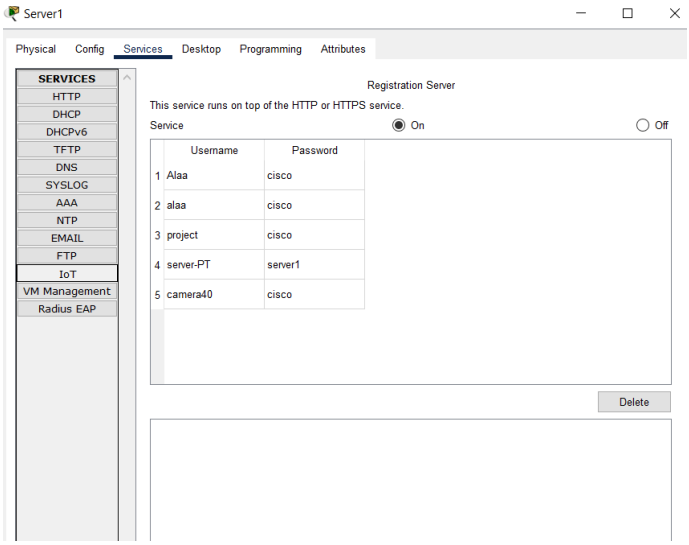


Figure 3.31. Network design evidence extracted from the Packet Tracer implementation report.

Password

- Objective: To secure server access, prevent unauthorized device connections, and ensure all IoT components are safely managed.

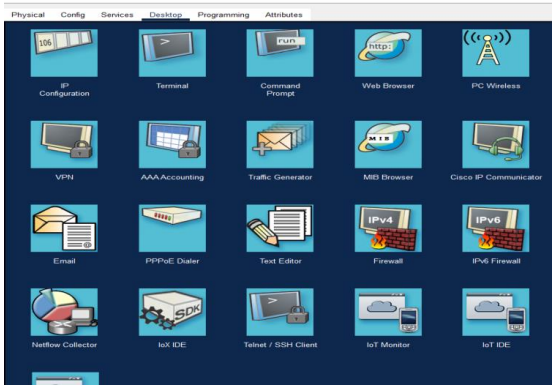


Figure 3.32. Network design evidence extracted from the Packet Tracer implementation report.

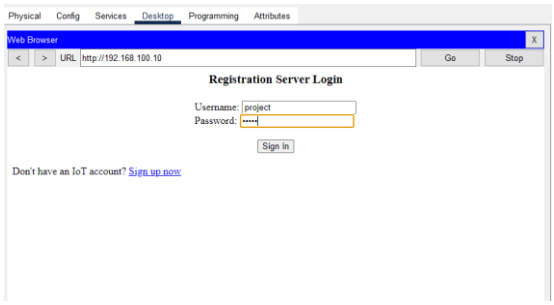


Figure 3.33. Network design evidence extracted from the Packet Tracer implementation report.

4.3 Registering the Webcam to the Server

Open the Webcam settings.

Navigate to the Config tab.

Under the IoT Server section, select Remote Server.

Enter the following credentials:

Username / Password.

Click Connect.

4.4 Registering the Motion Detector to the Server

Open the Motion Detector settings.

Navigate to the Config tab.

Select Remote Server and input the Server Address, Username, and Password.

Click Connect.

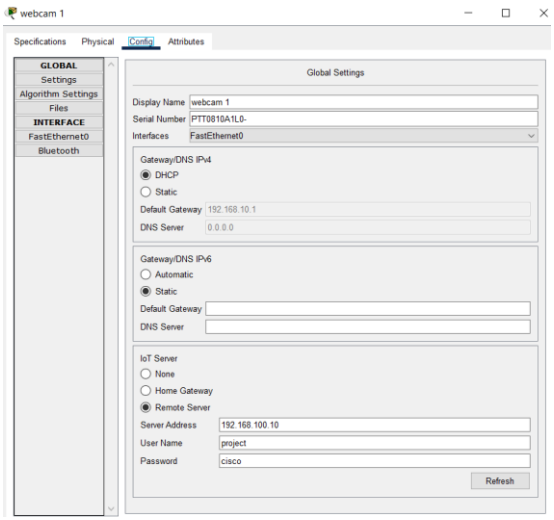


Figure 3.34. Network design evidence extracted from the Packet Tracer implementation report.

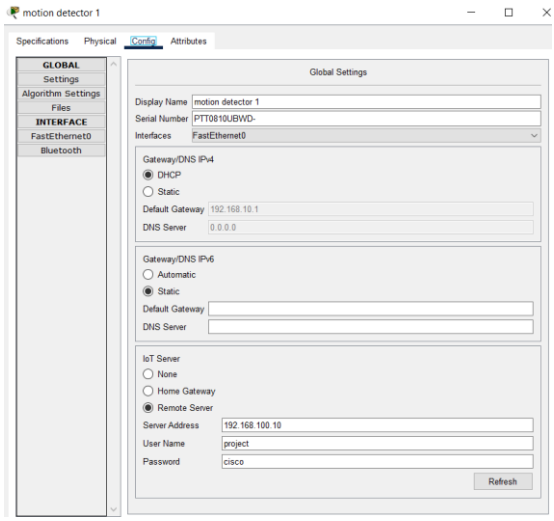


Figure 3.35. Network design evidence extracted from the Packet Tracer implementation report.

4.5 Objectives of Device–Server Integration

Centralized Control: Achieving unified management for all monitoring devices.

Enhanced Security: Securing connections using dedicated login credentials.

Access Control: Ensuring that no unauthorized devices can join the network.

System Integration: Enabling seamless interaction between the cameras and motion detectors to automate alerts and responses.

- To achieve a Smart Security System within the hospital, a logic-based scenario was designed. The system automatically activates the cameras when movement is detected by the Motion Detector, using the IoT Monitor & Conditions feature in Cisco Packet Tracer.
- Condition (Rule) Configuration Steps

5.1 Accessing the IoT Monitor

From the Server or a connected PC, open the Web Browser.

Enter the Server's IP Address and log in using your Username and Password.

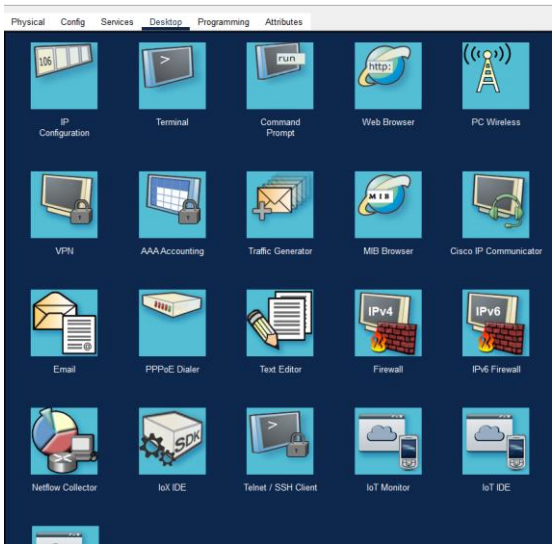


Figure 3.36. Network design evidence extracted from the Packet Tracer implementation report.

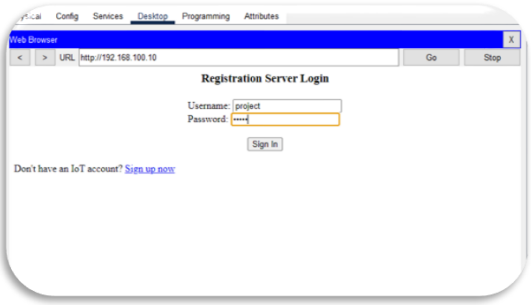


Figure 3.37. Network design evidence extracted from the Packet Tracer implementation report.

5.2 Accessing the Control Panel

After logging in, navigate to the IoT Monitor dashboard.

Select the Conditions (or Rules) tab.

5.3 Creating a New Condition (Add Condition)

Click Add and configure the logic as follows:

 Part 1: (IF – The Trigger)

Status: ON / Match Type: ALL

Select Device: Motion Detector

Set Condition: Motion Detector → On → Is → True

 Part 2: (THEN – The Action)

Select Action: Set

Select Device: Webcam

Set Status: Webcam → Status → Set To → On (True)

5.4 Saving Settings

Click OK / Save to apply the rule.

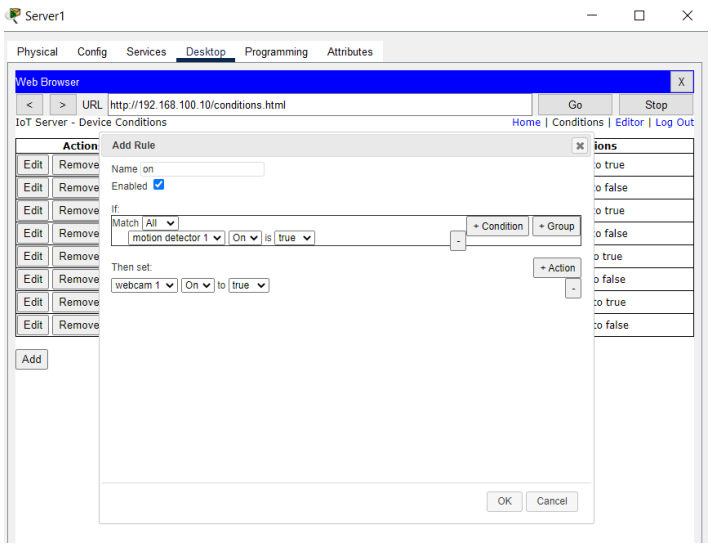


Figure 3.38. Network design evidence extracted from the Packet Tracer implementation report.

5.5 Scaling the Rules for All Devices To cover all monitored areas, the same steps were repeated for each pairing:

Condition 1: Motion Detector 1 → Camera 1

Condition 2: Motion Detector 2 → Camera 2

Condition 3: Motion Detector 3 → Camera 3

Condition 4: Motion Detector 4 → Camera 4

 Strategic Objectives of this Setup

Event-Driven Monitoring: Activating cameras only when necessary (upon motion detection).

Resource Optimization: Reducing unnecessary power consumption and storage usage.

Enhanced Efficiency: Improving the overall performance of the surveillance system.

Smart Response: Implementing a fully automated, intelligent security response system.

Final Result

Upon completing these steps, the system operates as follows:

Detection & Activation:

Whenever a person passes in front of a Motion Detector, the Camera is automatically triggered and starts recording immediately.

Idle State:

If no motion is detected, the Cameras remain in Standby/OFF mode

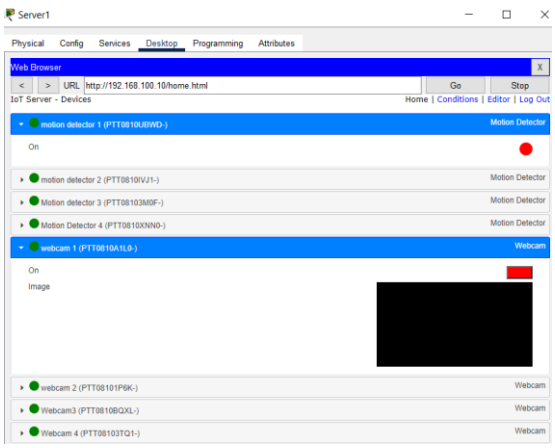


Figure 3.39. Network design evidence extracted from the Packet Tracer implementation report.

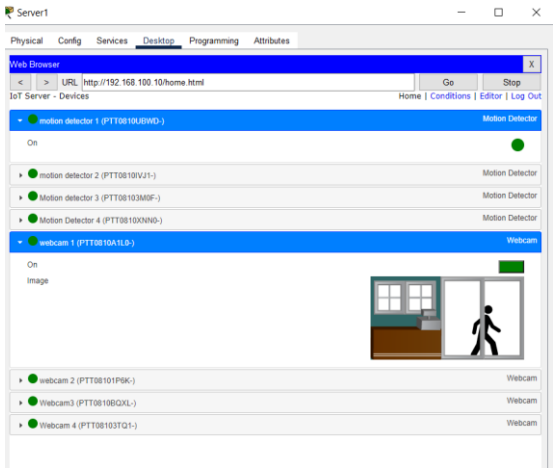


Figure 3.40. Network design evidence extracted from the Packet Tracer implementation report.

- **Wireless Network Configuration**

The wireless infrastructure was implemented using Generic Access Points and end-user devices (Laptops, PCs, and Tablets). The configuration process followed these steps:

1. Access Point (AP) Setup

PhysicalConnection:

The Access Point was connected to the main switch using a straight-through cable to integrate the wireless segment into the wired network.

SSIDConfiguration:

Under the Config tab, the SSID (Service Set Identifier) was defined to uniquely identify the wireless network.

Security&Authentication:

WPA2 encryption was configured to provide stronger wireless security and restrict unauthorized access to the wireless network.

PortStatus:

The wireless interface was set to On to begin broadcasting the signal.

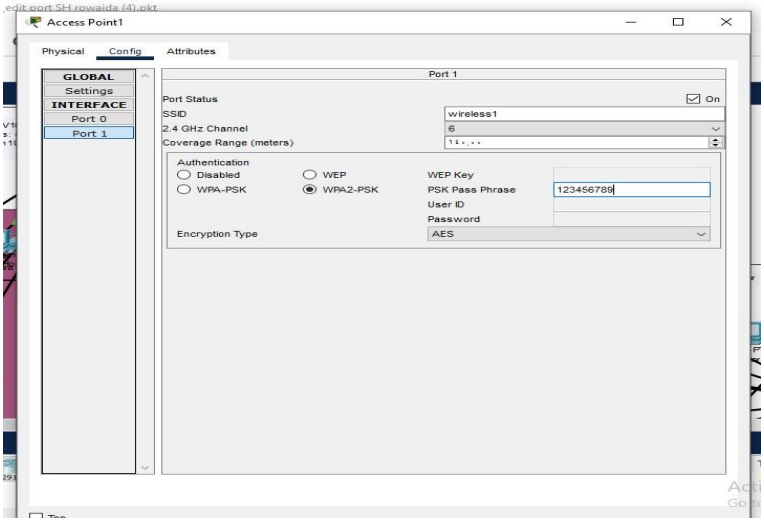


Figure 3.41. Network design evidence extracted from the Packet Tracer implementation report.

2. End-Device Configuration (PC/Laptop/Tablet)

Interface Activation:

For PCs/Laptops, the physical Ethernet module was replaced with a wireless module (such as WMP300N) when necessary, and the device was powered on.

ManualConfiguration:

In the Config tab, the SSID and authentication credentials (Password/Key) were configured to match the Access Point settings.

Wireless Desktop Utility:

Alternatively, the PC Wireless tool under the Desktop tab was used to:

Scan for available wireless networks.

Select the designated SSID from the Site Survey list.

Enter the security key to establish a secure connection with the Access Point

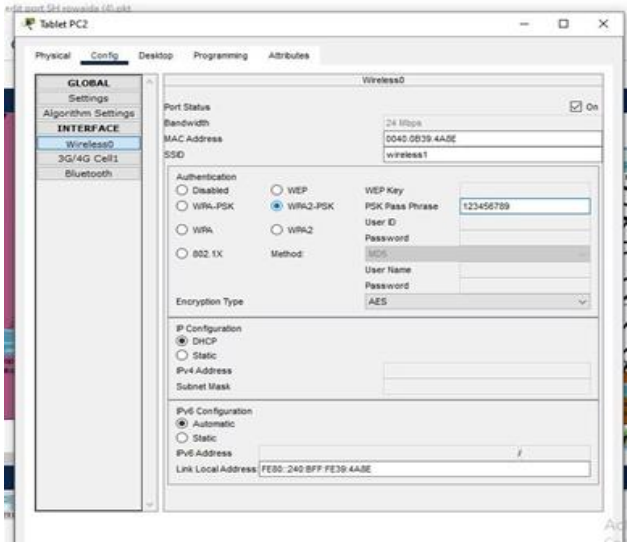


Figure 3.42. Network design evidence extracted from the Packet Tracer implementation report.

Figure 3.43. Network design evidence extracted from the Packet Tracer implementation report.

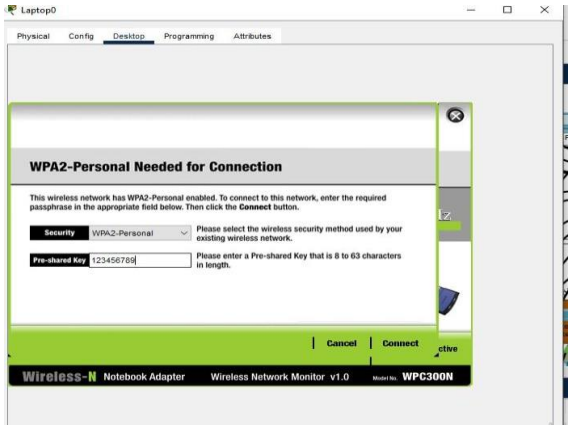


Figure 3.44. Network design evidence extracted from the Packet Tracer implementation report.

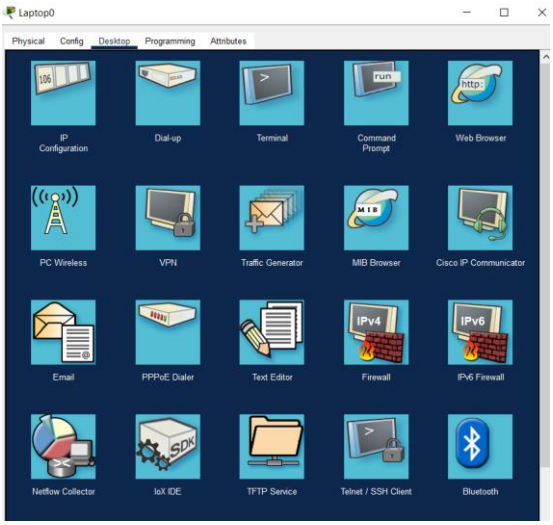


Figure 3.45. Network design evidence extracted from the Packet Tracer implementation report.

3. Connectivity Verification

After completing the configuration, the successful connection was verified by the appearance of the wireless link (dashed lines) in Packet Tracer and by performing a Ping test to ensure end-to-end communication between wireless clients and the rest of the network

- **Security** : -

  In our hospital network project, security was implemented to protect the network from unauthorized access and to ensure secure communication between devices. Wireless security was configured using **WPA2 authentication**, which provides stronger protection for the wireless network by requiring a secure password before any device can connect to the Access Point.

  The security configuration process included setting a unique **SSID** for the wireless network and creating a **WPA2 passphrase** to prevent unauthorized users from accessing the network. Only devices with the correct wireless password were able to connect successfully to the network.

  In addition, the security system helped maintain data privacy and improve network reliability inside the hospital environment. Implementing wireless security is important in hospital networks because it protects sensitive information, prevents network misuse, and ensures secure communication between laptops, tablets, and other connected devices.

3.9 SSH Security Configuration

1. Objective

The purpose of this phase was to secure remote management access for all network devices in the graduation project by replacing insecure Telnet access with Secure Shell (SSH).

SSH provides:

- Encrypted communication
- Secure remote administration

- User authentication
- Protection against password sniffing attacks

2. Devices Configured

The following devices were secured using SSH:

Device	Role
R1	Primary Router
R2	Backup Router
core_SW	Core Switch
SWV10	Access Switch

3. Security Problems Before Configuration

Before implementing SSH, the network had several security weaknesses:

- Telnet was enabled

Passwords were transmitted in plain text

No encrypted remote management

- Weak authentication mechanism

No secure administrator accounts

Example of old configuration:

This configuration allowed insecure Telnet access.

4. SSH Security Configuration Steps

Step 1 — Configure Device Hostname

Purpose:

- Assigns a unique identity to the device
- Required for SSH configuration

Step 2 — Configure Domain Name

Purpose:

- Required for RSA key generation
- Used by SSH encryption process

Step 3 — Generate RSA Encryption Keys

Purpose:

Enables SSH service

- Creates encryption keys for secure communication

Step 4 — Create Secure Administrator Account:

Purpose:

- Creates local authenticated user
- Uses encrypted secret password

Step 5 — Enable SSH on VTY Lines

Purpose:

- Disables Telnet
- Allows SSH access only
- Uses local username/password authentication

Step 6 — Secure Console Access

Purpose:

- Requires login authentication on console
- Automatically logs out inactive sessions after 5 minutes

Step 7 — Save Configuration

Purpose:

Saves all configurations permanently

5. Verification Commands

The following commands were used to verify successful SSH implementation:

Verify SSH Status

Verify VTY Configuration

Verify Username Configuration

6. SSH Connectivity Test

SSH remote access was tested using:

Authentication:

The test confirmed successful secure remote access between network devices.

7. Security Improvements Achieved

After implementing SSH:

✓Telnet was disabled

✓Remote access became encrypted

✓Authentication security improved

✓Unauthorized access risk reduced

✓Secure enterprise management implemented

8. Conclusion

Implementing SSH significantly improved the security level of the enterprise network project. The network devices are now managed using encrypted remote access with authenticated administrator accounts, following modern enterprise security standards

3.9.1 Port Security and DHCP Snooping Configuration

Port Security was implemented to secure the access layer by limiting connectivity to authorized devices only. The following settings were applied:

Max MAC Count: 1 (Only one device per port).

Sticky Learning: To dynamically learn and save the device's MAC address.

Violation Mode (Shutdown): If an unauthorized device connects, the port will automatically shut down and enter the err-disabled state to prevent access.

1. Securing the Address Assignment Service (DHCP Snooping)

The DHCP Snooping feature has been enabled to prevent Rogue DHCP Server attacks.

Important CLI commands are listed in Appendix D.

2. Port Security and Sticky MAC Address

The Port Security feature has been activated with Sticky MAC to protect ports from unauthorized access.

Important CLI commands are listed in Appendix D.

3. Access Point Port Configuration

The port configuration has been modified to allow multiple MAC addresses, as the Access Point (AP) serves more than one device.

Important CLI commands are listed in Appendix D.

4. Security Testing and Recovery Mechanism

When an unauthorized device is connected, the port is automatically shut down (placed into an error-disabled state).

To manually re-enable and restore the port, use the following commands:

Important CLI commands are listed in Appendix D.

3.10 Network Troubleshooting and Post-Mortem Report

Project: Hospital Network Infrastructure Design Subject: Analysis of Spanning Tree Protocol (STP) Anomalies and Root Bridge Election Date: June 2026

1. Executive Summary

During the deployment of the Access Layer switches (SWV10, SWV20, SWV30), we encountered conflicting visual indicators in the simulation software (Packet Tracer). While deploying a redundant topology without EtherChannel, we observed two distinct issues that looked identical (Green/Green lights on uplinks) but had completely different root causes: a software visual bug versus a critical configuration error (Root Bridge Election).

2. Issue #1: The Visual Anomaly (SWV30)

Affected Device: SWV30 (Access Layer)

2.1. The Scenario

After configuring SWV30 with redundant links to sw1 and sw2 (relying on STP for loop prevention), the network administrator noticed that both uplink indicators were GREEN.

Expected Behavior: In a redundant loop topology, STP should block one path. One light should be Green (Forwarding), and the other Orange (Blocking).

Initial Assumption: The administrator feared a configuration error or a broadcast storm loop.

2.2. Diagnosis & Analysis

We investigated the switch using the Command Line Interface (CLI) rather than relying on the GUI lights.

Command Used:

---------- ---- --- -------- ------- ----------

2.3. Conclusion

The configuration was actually correct. The port was logically blocked (BLK) by STP, ensuring network safety. The persistent green light was determined to be a Packet Tracer Visual Bug. No action was required for this switch.

3. Issue #2: The Critical Configuration Flaw (SWV10)

Affected Device: SWV10 (Access Layer)

3.1. The Scenario

Similar to SWV30, SWV10 showed two GREEN lights on its uplinks. Assuming this was the same visual bug, we initially disregarded it. However, network instability and DHCP delays suggested a deeper issue.

3.2. Diagnosis & Analysis

We ran the verification command to confirm the port states. Command Used:

Port Status: Both ports were in FWD (Forwarding) state. There was no blocking.

Critical Message: The output displayed: "This bridge is the root".

Root Cause Analysis: The small access switch (SWV10) had won the STP election and became the Root Bridge for VLAN 10.

Why? STP elects the Root Bridge based on the lowest Bridge ID (Priority + MAC Address). Since all switches had the default priority (32768), SWV10 won simply because it had a lower MAC address.

Impact: This forced all network traffic to flow toward the smallest switch, creating inefficient paths and potential bottlenecks.

4. The Solution: Deterministic Root Bridge Configuration

To fix Issue #2 and stabilize the network, we had to manually force the Core/Distribution switches to win the STP election.

4.1. Configuration on Primary Switch (sw1)

We configured sw1 to have the lowest priority, ensuring it becomes the Root Bridge.

! This sets priority to 24576 (Lower than default 32768)

4.2. Configuration on Secondary Switch (sw2)

We configured sw2 to act as a backup Root Bridge in case sw1 fails.

! This sets priority to 28672

4.3. Verification

After applying these commands, SWV10 lost its Root status.

Command: SWV10# show spanning-tree

3.11 VoIP Limitation

During the project development, we planned to implement VoIP technology to enable communication between different hospital departments. We already worked on configuring the VoIP system and assigning IP phones. However, we faced a technical problem where some devices were able to obtain IDs successfully while others could not connect properly.

After troubleshooting, which included replacing the Cisco 4331 router with a Cisco 2811 model and subsequently switching back to the Cisco 4331 to isolate the issue, and testing the configuration multiple times, we discovered that the issue was related to limitations in Cisco Packet Tracer, as the software did not fully support the VoIP functionality in our project environment the same way it works in real-life networks. Therefore, the VoIP feature was removed from the final project implementation to ensure network stability and proper performance of the remaining system components.

3.12 Network Design Final Conclusion

We successfully established a loop-free, redundant topology. The key lesson is that CLI verification (show spanning-tree) is the only reliable source of truth. Visual indicators in simulation software can be misleading. By manually configuring Root Bridge priorities, we ensured a deterministic and stable network traffic flow.

3.13 Future Network Expansion

- In the future, the hospital network can be expanded by adding more floors and departments to support a larger number of users and devices. Additional branches of the hospital can also be created in different

locations and connected together through the network to allow communication and data sharing between all branches.

- This development will improve network scalability, increase the efficiency of hospital management, and provide easier access to medical information across all connected hospital branches

Chapter 4: Web System and UI/UX Solution

4.1 Introduction

The ClinicEase website represents the main web interface of the software part of the project. It was developed to demonstrate how a healthcare organization can provide online services for guests, registered patients, and administrators through one connected system. The web solution supports public browsing, patient registration and sign-in, specialist exploration, doctor details, appointment booking, appointment tracking, patient scan viewing, and admin management pages.

This chapter focuses on the web system and UI/UX solution only. It explains the website objectives, technologies used, user roles, navigation structure, booking flow, scan workflow, dashboard management, and the design principles used to make the interface clear and suitable for a smart hospital case study.

4.2 Website Objectives

The website was designed to achieve the following objectives:

• Provide a clear public entry point where guests can understand the healthcare service and move to registration or booking.

• Allow patients to browse specialists and doctors, choose a suitable appointment date and time, and confirm appointment details.

• Give patients access to appointment categories such as upcoming, past, and canceled appointments.

• Provide a patient scan interface where uploaded medical scan records can be viewed and downloaded securely.

• Provide administrators with dashboard pages for monitoring appointments, patients, appointment status, and scan upload operations.

• Prepare the frontend structure for integration with the cloud backend through API Gateway, Lambda functions, DynamoDB, and S3.

4.3 Technologies Used

The website was implemented using standard frontend technologies suitable for a responsive healthcare prototype. HTML was used to structure the pages, CSS and Bootstrap were used for styling and layout, and JavaScript was used to handle user interaction, booking logic, navigation behavior, and local data simulation. During the prototype stage, localStorage was used to simulate saved user and appointment data. In the cloud-connected version, the same workflow is prepared to communicate with backend APIs for authentication, appointment storage, appointment retrieval, appointment status updates, and scan management.

4.4 Website Structure and User Roles

The web system was divided into three main user roles: guest, patient, and administrator. This role-based design makes the system easier to understand and prevents users from seeing pages that are not related to their responsibilities.

Guest: The guest can open the landing page, read information about the healthcare service, view available specialties, and move to sign-in or sign-up when a protected action is required.

Patient: The patient can sign in, browse specialists, view doctor details, book appointments, check appointment history, view medical scans, and open the profile page.

Administrator: The administrator can access the dashboard, view statistics, manage appointment records, review patient information, update appointment status, and upload medical scans for specific patients.

4.5 Guest Flow

The guest flow starts from the public landing page. This page introduces the ClinicEase concept and guides the visitor toward the main actions of the website. A guest can browse sections such as the home page, about page, specialists page, and booking call-to-action. The purpose of this flow is to convert a visitor into a registered patient by making the service easy to understand before asking the user to create an account. From a UI/UX perspective, the guest flow uses simple navigation, clear buttons, service cards, and a healthcare visual style. The flow avoids overwhelming the user with technical details and focuses on the main decision: choosing a specialist and starting the booking process.

4.6 Patient Flow

The patient flow starts after account access. Once signed in, the patient reaches a logged-in home page with navigation links to Home, Specialists, My Appointments, Patient Scan, and My Profile. This structure gives the patient quick access to the most important healthcare actions without requiring complex navigation. The patient journey moves from specialist selection to doctor selection, then to doctor details and booking confirmation. The patient chooses a specialization, opens the list of related doctors, reviews the selected doctor information, selects a day and time, enters the required personal information, and confirms the appointment. The workflow is intentionally divided into small steps to reduce user confusion and make the booking experience straightforward.

4.7 Appointment Booking Workflow

The appointment booking workflow is the central function of the web system. It connects the patient interface with the administrative side of the project. The user first selects the required specialization, then chooses a doctor, reviews the doctor details, and selects an available appointment slot. The confirmation page collects key information such as national ID, full name, and email before submitting the booking.

In the frontend prototype, booking data can be stored using browser localStorage to simulate persistence during development. In the cloud-connected version, this workflow is mapped to the book-appointment API endpoint. The frontend sends the appointment details to API Gateway, which invokes a Lambda function to validate and store the appointment record in DynamoDB. This makes the web interface ready for real backend integration while keeping the user experience simple.

4.8 Patient Appointment and Scan Interfaces

The My Appointments page gives the patient a clear view of appointment history and status. Appointments are organized into categories such as upcoming, past, and canceled. This improves usability because the patient does not need to search through all records manually.

The patient scan interface adds an important healthcare document-management feature. Instead of limiting the website to appointment booking only, the system allows patients to access scan records uploaded by the administrator. In the cloud solution, scan files are stored in S3 while scan metadata is stored in DynamoDB. The frontend can request secure download links when the patient needs to access a scan file.

4.9 Admin Dashboard and Management Flow

The admin dashboard represents the operational management side of the website. It provides a quick overview of total appointments, patient records, appointment activity, and system status. The administrator can use dedicated pages to review appointments, update appointment status, view patients, and upload scans.

This flow is important because a healthcare system requires more than patient-facing pages. Hospital staff need tools to monitor and manage the data submitted by patients. The admin pages turn the website from a simple booking interface into a complete management prototype.

4.10 UI/UX Design Principles

The UI/UX design follows a clean healthcare style. The interface uses light backgrounds, blue accents, rounded cards, readable typography, clear spacing, and direct call-to-action buttons. The layout separates the website into understandable actions: browse, authenticate, select specialist, select doctor, choose appointment, confirm booking, manage appointments, upload scans, and view scans.

The design also applies role-based navigation. Guest pages focus on discovery and conversion, patient pages focus on booking and personal records, and admin pages focus on management and monitoring. This separation improves clarity and helps each user complete tasks with fewer steps.

4.11 Website Screens Explanation

The following screenshots document the main interfaces of the ClinicEase website. Each figure supports a specific part of the web workflow and shows how the UI/UX design was translated into practical screens.

Figure 4.1. Admin patients records page.

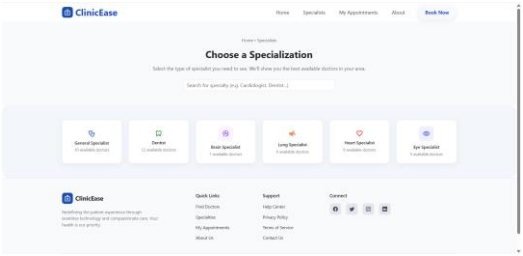


Figure 4.2. Medical specialist selection page.

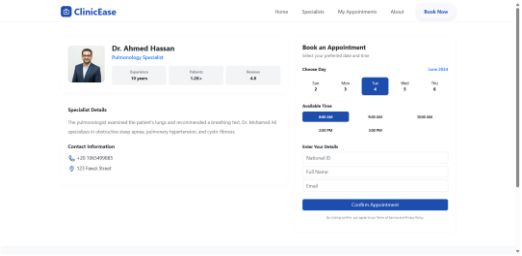


Figure 4.3. Doctor details and time selection page.

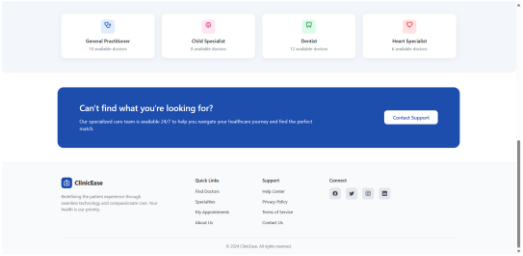


Figure 4.4. Landing page service call-to-action.

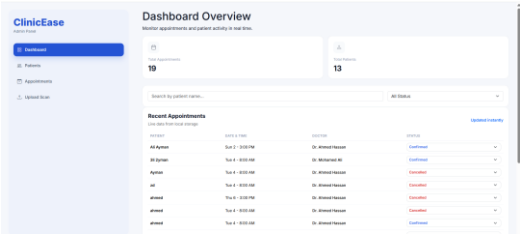


Figure 4.5. Admin dashboard overview page.

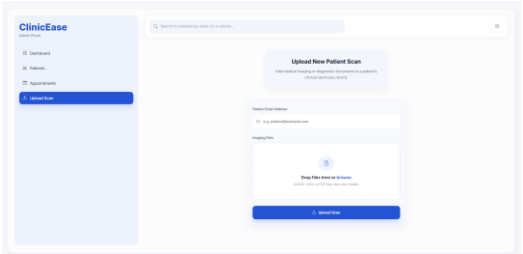


Figure 4.6. Admin scan upload page.

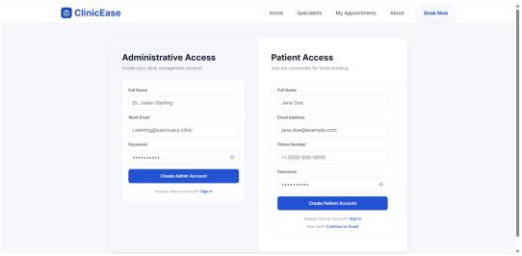


Figure 4.7. Admin and patient authentication page.

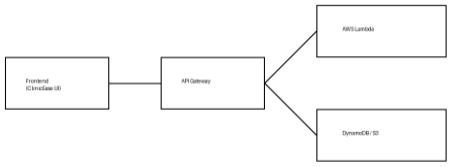


Figure 4.8. Web-to-cloud architecture diagram.

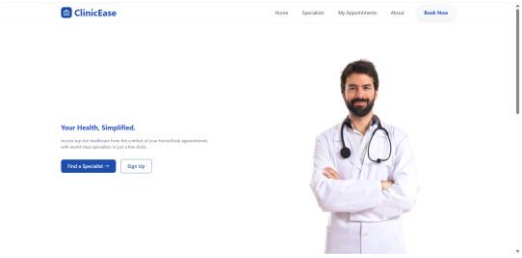


Figure 4.9. ClinicEase public landing page.

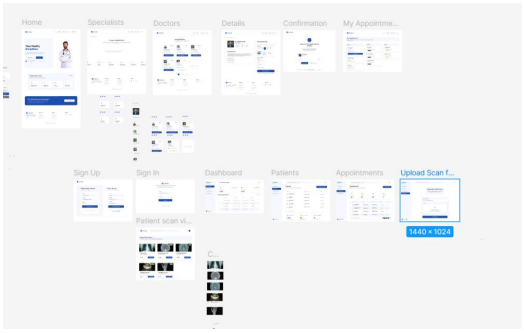


Figure 4.10. Website UI/UX flow overview.

Screen-by-Screen Explanation

Figures 4.1 and 4.5 represent the administrative management side. The patients page displays patient records in a searchable table, while the dashboard overview summarizes system activity and gives the administrator a quick

operational view. These screens prove that the website is not only a patient booking interface, but also includes management functionality for hospital staff.

Figures 4.2, 4.3, and 4.10 represent the patient journey. The specialist selection screen helps the patient choose the required department, the doctor details page provides doctor information and appointment slots, and the UI/UX flow overview shows how the patient moves through the main screens. This sequence demonstrates a clear and guided booking experience.

Figures 4.4, 4.7, and 4.9 represent access and public navigation. The landing page introduces the website to guests, while the authentication interface separates patient access from administrative access. This supports role-based interaction and keeps protected functions behind account access.

Figures 4.6 and 4.8 connect the web system to the cloud solution. The scan upload page supports healthcare document management, and the architecture diagram shows how the frontend communicates with API Gateway, Lambda functions, DynamoDB, and S3. These screens demonstrate that the web interface is prepared for backend and cloud integration.

4.12 Chapter Summary

This chapter presented the ClinicEase web system as a complete UI/UX and frontend implementation. The website supports guest browsing, patient booking, appointment tracking, scan viewing, and admin management. The chapter also explained how the frontend prototype is prepared for cloud backend integration, making it an essential part of the overall integrated software engineering solution.

Chapter 5: Mobile Application Solution

5.1 Mobile App Objectives

The ClinicEase mobile application provides a patient-friendly booking experience on mobile devices. Its objective is to allow patients to search for specialists, view doctor details, select appointment dates and times, confirm booking information, receive success feedback, and access appointment and notification tabs. The mobile app supports the same healthcare concept as the website, but with a mobile-first interface.

5.2 Flutter Technical Stack

Component	Technology / Method
Framework	Flutter
Programming Language	Dart
State Management	BLoC / Cubit
Responsiveness	Flutter ScreenUtil with design size 375x812
Localization	Flutter L10n with RTL/LTR support
Animations	Lottie animations
Notifications	Local notification system for confirmations and reminders

5.3 Application Architecture

The application employs a unidirectional data flow architecture powered by the BLoC pattern. When a patient performs an action, such as submitting appointment details, the UI directly calls a function within the Cubit. The Cubit processes the request within the simulated local data layer and instantly emits a 'success' or 'error' state. This architecture strictly separates the user interface from business logic, ensuring scalability and maintainability

5.4 Screen-by-Screen Description

The mobile application contains several screens. The splash screen introduces the brand and performs initial setup. The home screen acts as the patient dashboard and adapts to new or returning users. The specialization screen presents available medical departments. The doctors screen lists doctors under the selected specialization. The doctor details screen contains profile information and a schedule picker. The confirmation screen validates patient data before booking. The success screen displays positive feedback and triggers a notification. Appointment tabs show upcoming, past, and empty states. Notification tabs show reminders and system alerts.

5.5 Booking Workflow

1. The patient opens the application and selects a specialization.
2. The application displays doctors related to that specialization.
3. The patient opens the doctor details screen and selects an available date and time.
4. The patient enters required confirmation data.
5. The Cubit processes the request and emits success, or error states.
6. The application navigates to the success screen and triggers a local notification.

5.6 Core Application Features

5.6.1 Advanced Search Mechanism

The application implements a real-time, query-based search mechanism on the client side. This feature captures user input dynamically to filter the local list of medical specializations and available doctors instantly. By utilizing the active Cubit state management, the user interface updates seamlessly to display matching results ensuring a fast and responsive user experience.

5.6.2 Bilingual Support (Localization)

The mobile app supports a bilingual user experience through English and Arabic localization. Full RTL/LTR layout switching ensures that the interface adapts smoothly to the selected language, making the application highly suitable, accessible, and user-friendly for both Arabic and English-speaking patients.

5.6.3 Local Notification System

Automated local notifications are integrated into the system to improve patient engagement and tracking. The application triggers immediate alerts and push-style notifications on the user's device upon successful appointment confirmations, acting as persistent reminders for upcoming clinic visits.

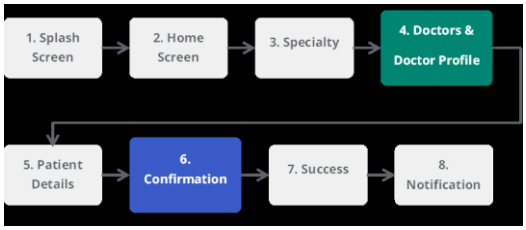


Figure 5.1: Application flow diagram illustrating the seamless user journey from the Splash Screen through the booking process to the final Success and Notification stages.

Figure 5.2: The Splash Screen serves as the application's initial entry point, establishing brand identity while awaiting critical background initializations like ScreenUtil and localization.

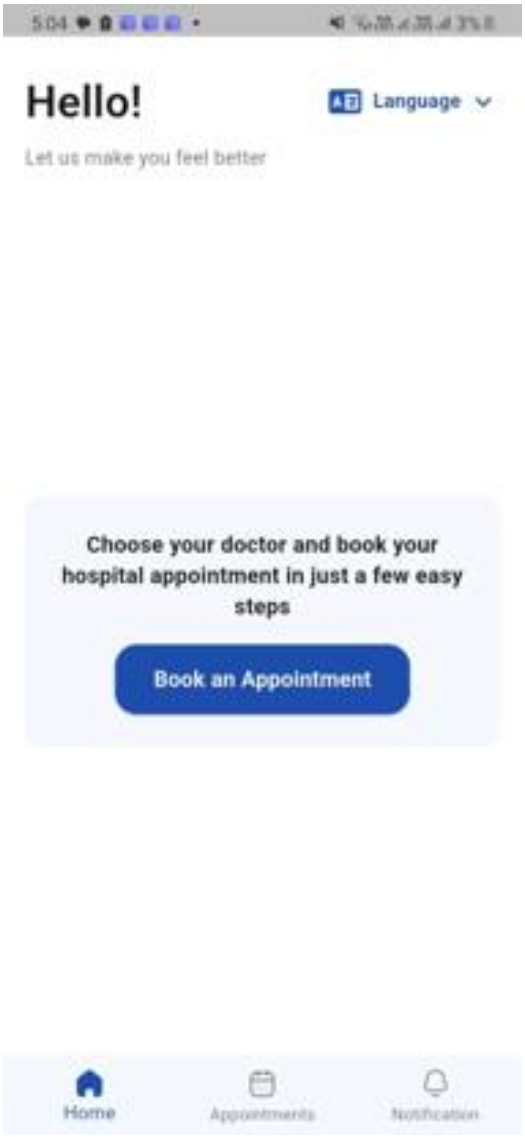


Figure 5.3: Home Screen (Default State) prioritizing accessibility for new users, featuring a prominent booking prompt to minimize cognitive load and encourage exploration.

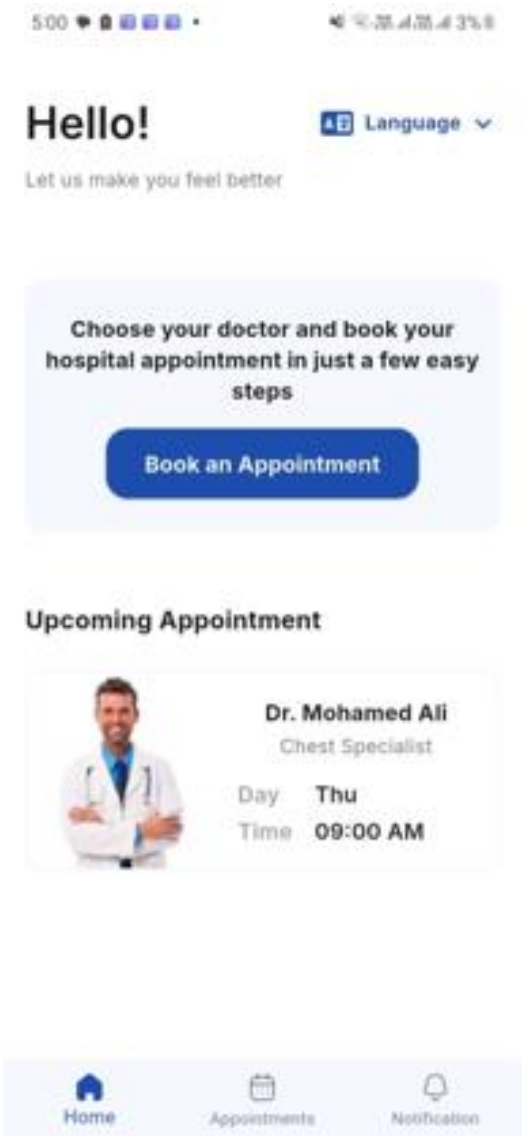


Figure 5.4: Home Screen (Active State) dynamically adapting to returning users by displaying personalized greetings and highlighting upcoming scheduled appointments for quick access.

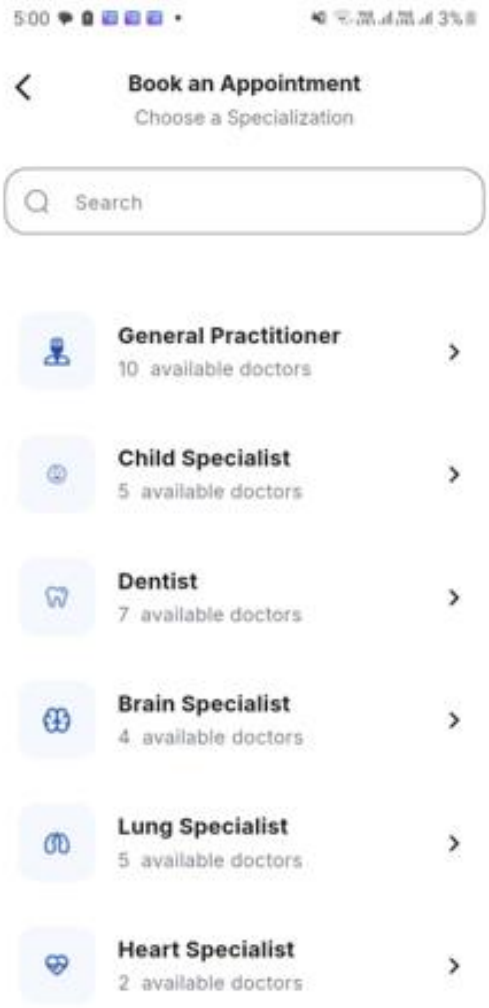


Figure 5.5: Specialization Screen functioning as a comprehensive directory for medical departments, utilizing a clear list-based layout for intuitive patient navigation.

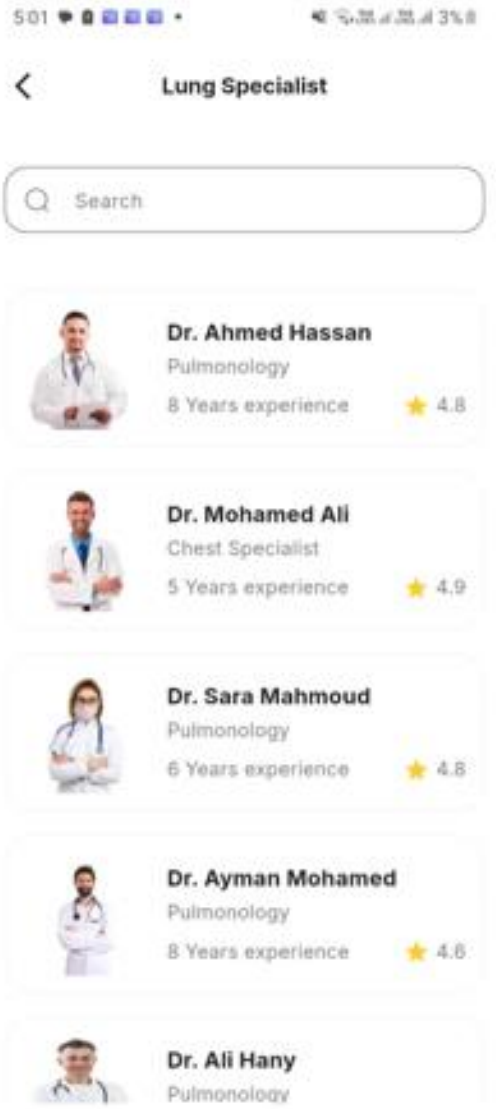


Figure 5.6: Doctors Screen presenting a filtered list of healthcare professionals based on selected specialties, showcasing essential metadata such as experience and patient ratings.

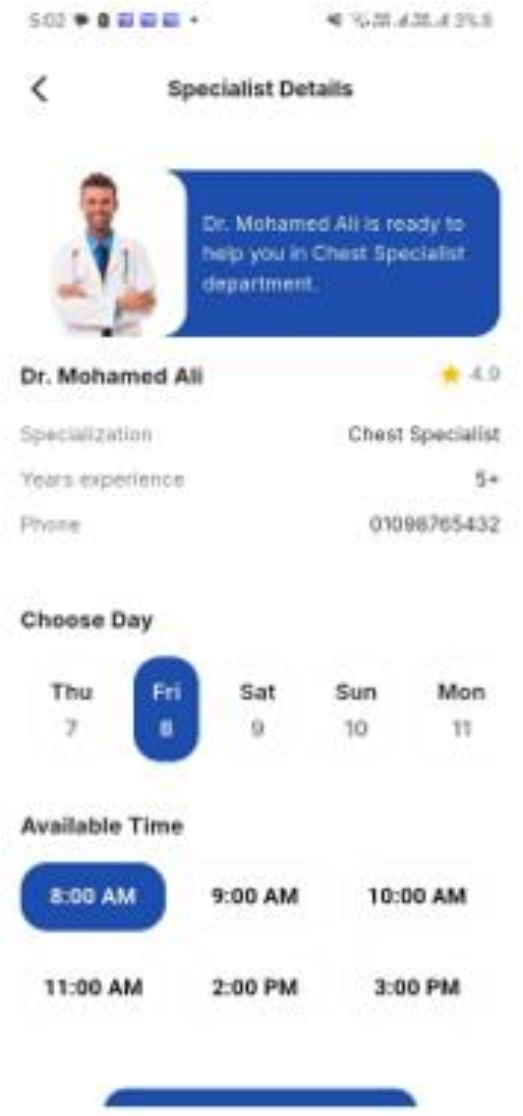


Figure 5.7: Doctor Details Screen providing an in-depth specialist profile alongside an interactive, horizontally scrollable schedule picker managed by the state Cubit.

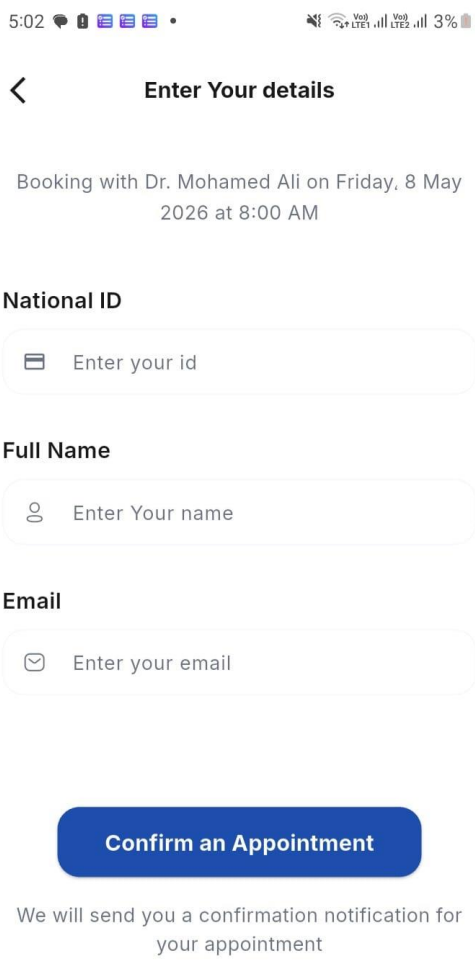


Figure 5.8: Confirmation Screen acting as a final validation gate, requiring patients to input personal identification details securely before committing to the appointment.

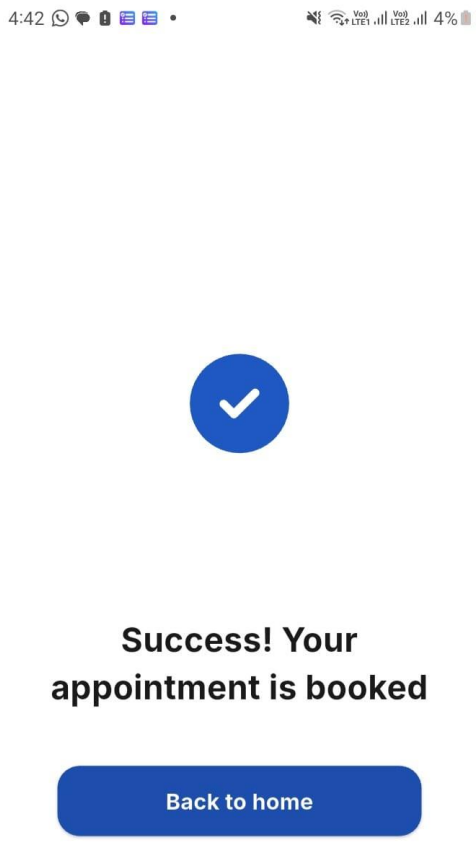


Figure 5.9: Success Screen providing immediate positive reinforcement, visually confirming the successful completion of the booking process.

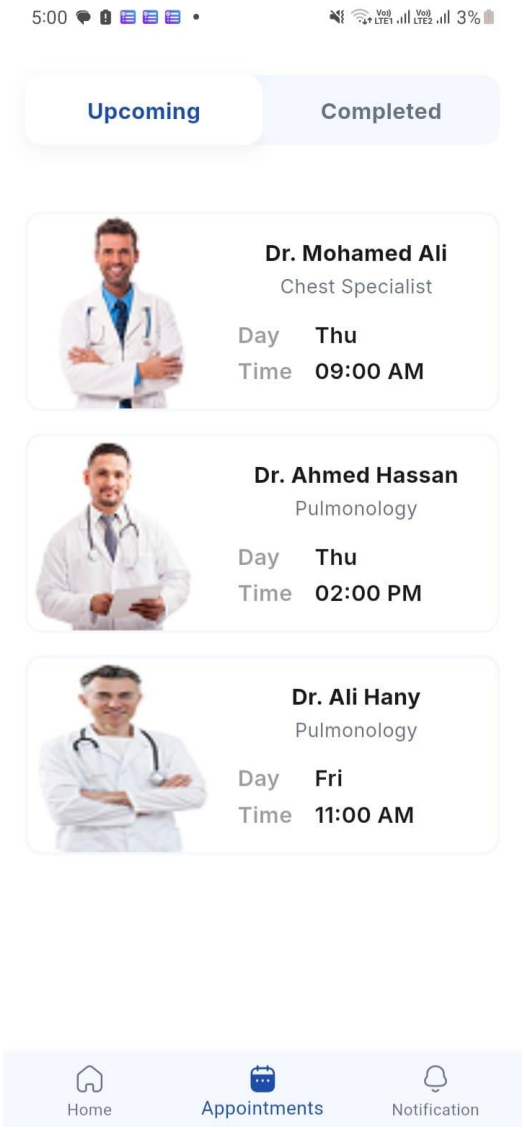


Figure 5.10: Appointments Tab (Upcoming) displaying a dynamic, reactively managed list of active scheduled visits, ensuring real-time accuracy for the patient's schedule.

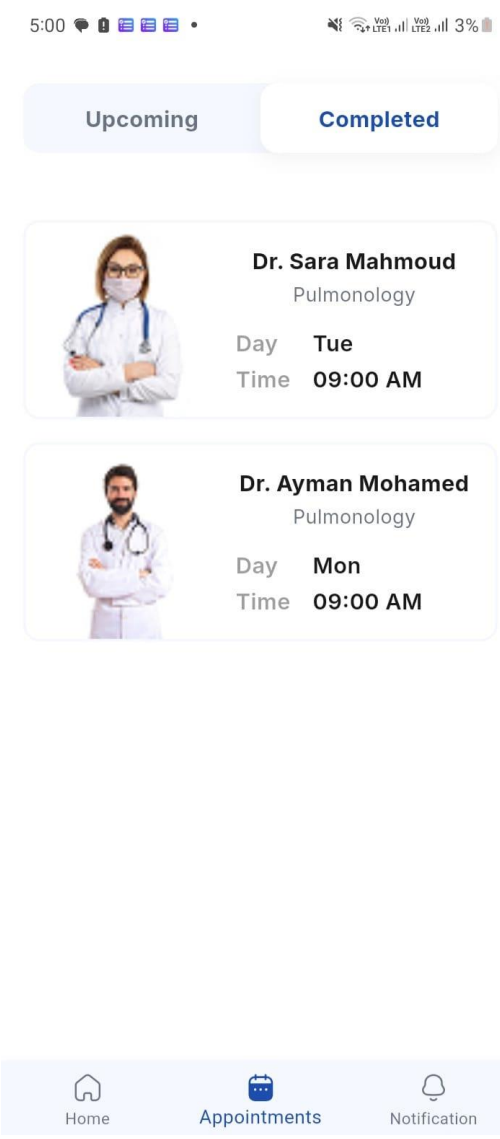


Figure 5.11: Appointments Tab (Completed) offering a structured historical log of past medical visits, allowing patients to review their consultation history seamlessly.

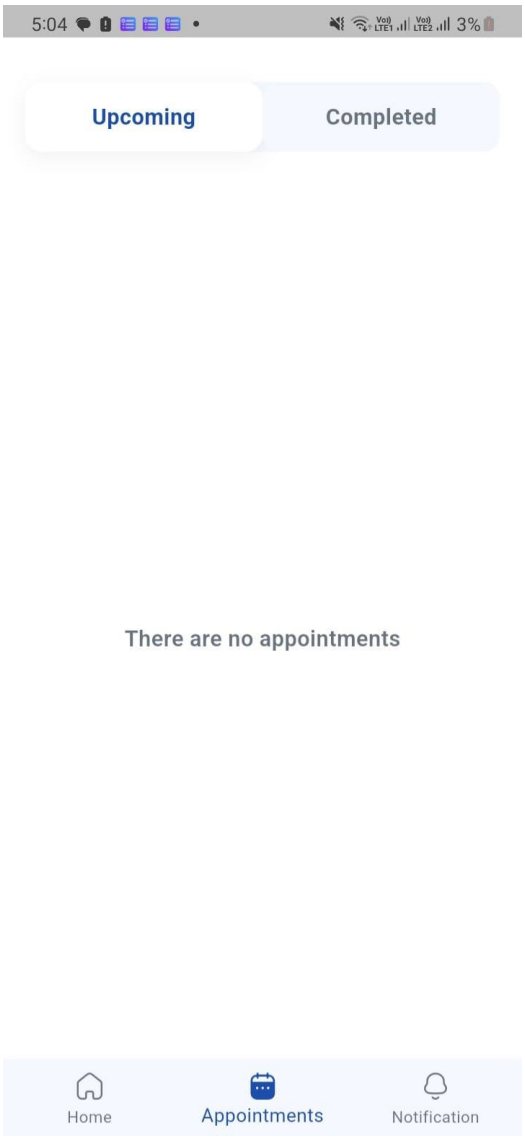


Figure 5.12: Appointments Tab (Empty State) featuring an intuitive, user-friendly interface that encourages first-time bookings when no prior appointment history exists.

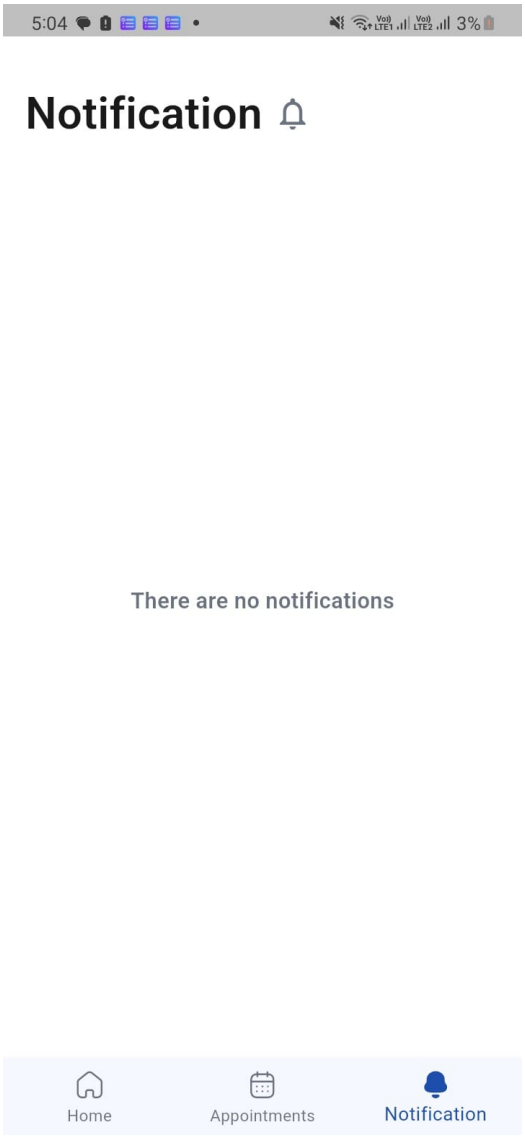


Figure 5.13: Notifications Tab (Empty State) demonstrating a clean, visually pleasing UI that prevents the screen from appearing broken when no alerts are present.

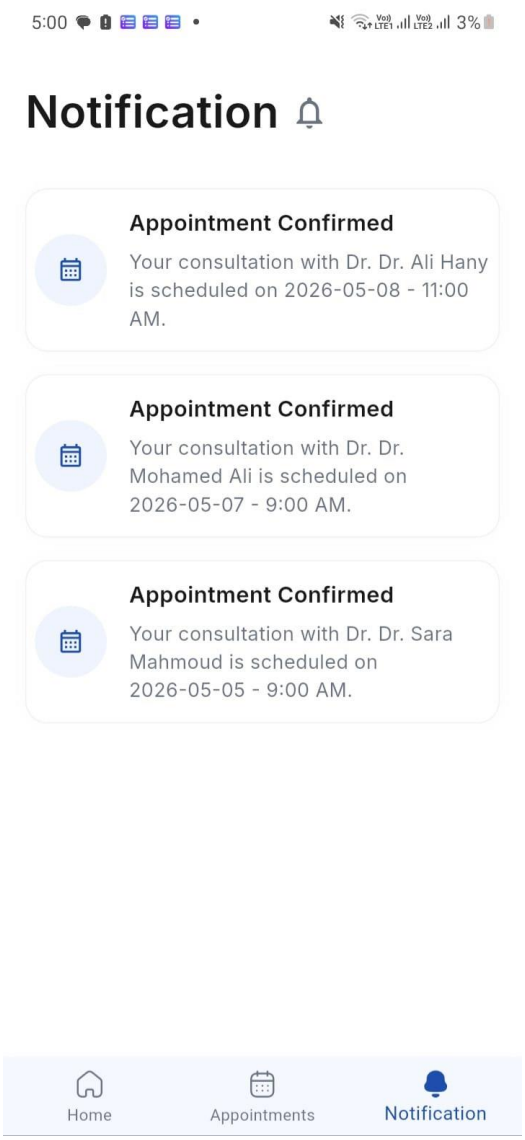


Figure 5.14: Notifications Tab (Active State) serving as a centralized inbox for critical patient alerts, including appointment confirmations and visit reminders.

Chapter 6: Cloud and Backend Solution

This chapter presents the complete cloud and backend implementation of the ClinicEase system. It explains why the cloud layer was needed, how AWS services were used, how the frontend communicates with the backend, how medical scans and appointments are stored, the Lambda implementation used in the project, the problems faced during implementation, and the expected real-world cost if the solution is deployed for a clinic or hospital.

Chapter Purpose

The cloud part converted ClinicEase from a static healthcare interface into a connected healthcare platform where patient accounts, appointments, admin actions, medical scan metadata, and future sensor readings can be accessed securely from different devices.

6.1 Cloud Migration Objective

The first version of the website could display pages and simulate user interaction, but healthcare systems require real data storage, role-based access, and secure file management. Therefore, the cloud backend was introduced to move the project from local browser behavior into a practical distributed system. The objective was not only to store data, but also to create a foundation that can support real hospital workflows in the future.

- Store patient accounts and authentication data in DynamoDB instead of relying on frontend-only logic.
- Store appointment records in a central database so both patients and admins can access the same data.
- Allow the admin dashboard to monitor appointments and update appointment statuses in real time.
- Upload and retrieve medical scan files through private S3 storage using secure presigned URLs.
- Prepare the system for future hardware integration, where ESP32/Arduino health readings can be sent to the same backend.
- Reduce the need for local servers, manual backups, and hardware maintenance by using managed AWS services.

6.2 Implemented AWS Cloud Architecture

The implemented design follows a serverless architecture. The frontend sends HTTPS requests to Amazon API Gateway. API Gateway acts as the secure public entry point and invokes Python Lambda functions. Each Lambda function performs one clear backend task, such as signing in a patient, booking an appointment, updating a status, generating a scan upload URL, or saving scan metadata. DynamoDB stores structured records, while Amazon S3 stores heavy medical files such as images and PDF scans.

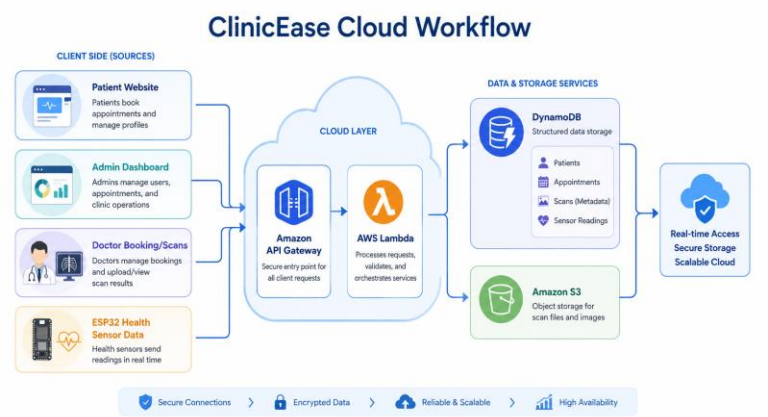


Figure 6.1. ClinicEase cloud workflow showing frontend sources, API Gateway, Lambda, DynamoDB, S3, and secure cloud outcomes.

Layer	Implemented Component	Role in ClinicEase
Frontend	HTML, CSS, Bootstrap, JavaScript	Patient portal, booking pages, profile page, scan viewer, and admin dashboard.
API Layer	Amazon API Gateway REST APIs	Receives HTTPS POST requests and routes them to Lambda functions.
Compute Layer	AWS Lambda – Python 3.11	Runs backend logic without managing a server.
Database Layer	Amazon DynamoDB	Stores patients, admins, appointments, scan metadata, and sensor readings.
File Storage Layer	Amazon S3	Stores scan files and medical imaging documents privately.
Security Layer	IAM roles, CORS, presigned URLs	Controls permissions and avoids exposing the S3 bucket publicly.

Table 6.1. Main AWS cloud layers used in ClinicEase.

6.3 AWS Resources Created for the Project

The AWS account contained four DynamoDB tables, ten main Lambda functions, and two REST APIs in API Gateway. Splitting resources by responsibility made the backend easier to debug and allowed the frontend to call only the service it needed.

Resource Type	Resource Name / ID	Purpose
DynamoDB Table	ClinicEaseAdmins	Stores admin account data such as admin@clinicease.com and role=admin.
DynamoDB Table	ClinicEasePatients	Stores patient registration data, password hashes, phone numbers, and role=patient.
DynamoDB Table	ClinicEaseAppointments	Stores patient bookings, doctor details, appointment date/time, and status.
DynamoDB Table	ClinicEaseScans	Stores scan metadata such as scan_id, patient_email, file_name, file_key, content_type, and uploaded_at.
S3 Bucket	clinicease-scans-mohamed	Stores actual scan files privately. DynamoDB stores only metadata and S3 keys.
API Gateway	cbh42xu5yf	Contains booking-related endpoints: /book-appointment and /my-appointments.
API Gateway	lqum8loiu7	Contains signup, signin, dashboard, scan upload/download, and status endpoints.
Lambda Runtime	Python 3.11	All backend functions were implemented as Python Lambda functions.

Table 6.2. Implemented cloud resources and their responsibilities.

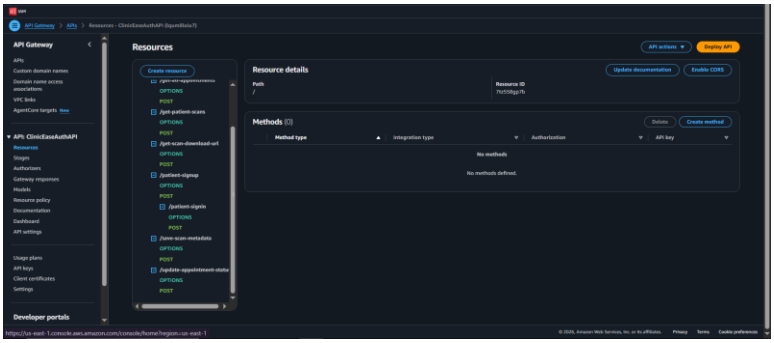


Figure 6.2. API Gateway resources showing ClinicEase endpoints connected to the backend Lambda services.

6.4 DynamoDB Data Model

DynamoDB was selected because the project data is document-style and highly suitable for key-value access. Patient records are retrieved by email, appointment records are retrieved by appointment_id or filtered by patient_email, and scan metadata can be queried by patient_email using a secondary index. This is simpler and cheaper for a prototype than managing a relational database server.

Table	Primary Key	Important Attributes	Used By
ClinicEaseAdmins	email	full_name, password_hash, role	Admin login
ClinicEasePatients	email	full_name, phone, password_hash, role, created_at	Patient signup and signin
ClinicEaseAppointments	appointment_id	patient_name, patient_email, national_id, doctor_name, doctor_specialty, day, time, appointment_date, status, created_at	Booking system and admin dashboard
ClinicEaseScans	scan_id	patient_email, uploaded_by, file_name, file_key, content_type, uploaded_at	Admin scan upload and patient scan viewer
patients	patient_id + timestamp	temperature, heart_rate, oxygen, status	Future hardware sensor readings from ESP32/Arduino

Table 6.3. DynamoDB tables and data fields.

6.5 API Gateway Endpoint Design

The frontend communicates with the backend through REST endpoints. Each endpoint uses POST because the frontend sends JSON request bodies. OPTIONS methods were added for CORS support so the browser can safely call the API from the website.

Endpoint	API ID	Frontend Source	Lambda Function	Purpose
/patient-signup	lqum8loiu7	signup.html	clinicease-patient-signup	Creates patient account.
/patient-signin	lqum8loiu7	signin.html	clinicease-patient-signin	Authenticates admin or patient.
/book-appointment	cbh42xu5yf	hospital.js	clinicease-book-appointment	Creates new appointment.
/my-appointments	cbh42xu5yf	hospital.js	clinicease-get-my-appointments	Loads patient appointments.
/get-all-appointments	lqum8loiu7	dashboard.js	clinicease-get-all-appointments	Loads admin dashboard appointment data.
/update-appointment-status	lqum8loiu7	dashboard.js	clinicease-update-appointment-status	Updates appointment status.
/generate-scan-upload-url	lqum8loiu7	dashboard.js	clinicease-generate-scan-upload-url	Creates S3 upload URL.
/save-scan-metadata	lqum8loiu7	dashboard.js	clinicease-save-scan-metadata	Saves scan metadata.
/get-patient-scans	lqum8loiu7	patient-scans.js	clinicease-get-patient-scans	Returns patient scan list.
/get-scan-download-url	lqum8loiu7	patient-scans.js	clinicease-get-scan-download-url	Creates secure scan download URL.
/data	i1u3kafk31 or future API	ESP32 / Arduino	save-patient-reading	Stores hardware sensor readings.

Table 6.4. API Gateway endpoints and their Lambda integrations.

6.6 Backend Workflows

6.6.1 Patient Signup and Signin Workflow

7. The patient enters full name, email, phone number, and password in signup.html.
8. The frontend sends a JSON request to /patient-signup.
9. Lambda validates the request, hashes the password, and saves the patient in ClinicEasePatients.
10. During signin, the frontend sends email and password to /patient-signin.
11. Lambda checks both admin and patient tables and returns a role-based response.
12. The frontend stores the returned user object in localStorage for the prototype session.

6.6.2 Appointment Booking Workflow

Appointment booking is the main patient workflow. The patient selects a doctor, day, and time. The frontend sends the booking details to Lambda. Lambda writes a new item in ClinicEaseAppointments with status set to Pending. The admin can later change the status to Confirmed, Cancelled, or Completed from the dashboard.

Step	Action	Cloud Service
1	Patient fills booking form and confirms appointment.	Frontend JavaScript
2	Request is sent to /book-appointment.	API Gateway
3	Appointment data is validated and appointment_id is generated.	AWS Lambda
4	Appointment is stored with status = Pending.	DynamoDB
5	Admin dashboard reads and updates status.	API Gateway + Lambda + DynamoDB
6	Patient appointment page separates records into upcoming, past, and cancelled.	Frontend + Cloud Data

Table 6.5. Appointment booking workflow.

6.6.3 Medical Scan Upload and Download Workflow

Medical scans can be large and should not be transferred through Lambda as raw file bytes. The project uses S3 presigned URLs. The admin first requests an upload URL, then the browser uploads the file directly to S3. After the upload succeeds, the metadata is saved in DynamoDB. The patient viewer loads metadata and asks for a temporary download URL only when the patient clicks Download.

- The scan file remains in a private S3 bucket.
- The database stores only metadata and file_key, not the full file bytes.
- Presigned URLs expire after a short time, reducing the risk of public exposure.
- The patient can view only scans linked to the signed-in patient email.

6.6.4 Hardware Sensor-to-Cloud Workflow

The hardware extension connects ESP32/Arduino sensor readings to the cloud. The embedded board reads heart rate and oxygen from MAX30102 and sends the JSON payload to API Gateway. Temperature can be simulated in the prototype if a real temperature sensor is not connected. Lambda stores the reading in DynamoDB and classifies it as normal or alert.

JSON Field	Source	Example	Cloud Usage
patient_id	ESP32 code / device setting	P001	Partition key for sensor readings.
temperature	Random value or future DHT22 sensor	37.1	Used for alert decision.
heart_rate	MAX30102 / project logic	82	Used for alert decision.
oxygen	MAX30102 SpO2 reading	97	Used for alert decision.
status	Lambda calculated value	normal / alert	Stored with reading for dashboard use.

Table 6.6. Hardware sensor data sent to the cloud.

6.7 Implementation Challenges and Fixes

The cloud implementation was not only a configuration task. Several practical problems appeared during development, especially because the website, APIs, Lambda functions, DynamoDB tables, and S3 bucket all had to work together. The following table summarizes the main problems and how they were handled.

Challenge	Cause	Solution / Decision
Appointment appeared in Past instead of Upcoming	The booking page contained a static month/year value such as June 2024 while the actual project year was later.	The appointment date should be generated dynamically from the current year, and the patient page should move appointments to Past only after the appointment day has passed.
ClinicEaseScans table showed no items	The file upload alone does not create a database record. Metadata must be saved after the S3 upload succeeds.	The scan flow was divided into two steps: upload to S3, then call save-scan-metadata to write scan_id, patient_email, file_key, and uploaded_at.
API Gateway root showed Methods (0)	The root resource / was selected in AWS Console instead of the actual child resource.	The methods were verified under each endpoint such as /patient-signin, /book-appointment, and /save-scan-metadata.
Two API Gateway base URLs existed	Booking endpoints were created in cbh42xu5yf while scans/auth/dashboard endpoints were created in lqum8loiu7.	The frontend was updated carefully so each JavaScript file calls the correct base URL. A future improvement is to merge endpoints under one API.
CORS browser errors	Browser preflight requests require OPTIONS method and CORS headers.	Each Lambda response includes Access-Control-Allow-Origin and the API Gateway resources include OPTIONS methods.
S3 files should not be public	Medical scans are sensitive and should not be exposed through public URLs.	The S3 bucket remains private, and temporary presigned URLs are generated for upload and download.
DynamoDB query by patient email	ClinicEaseScans uses scan_id as the main key but the patient viewer needs patient_email lookups.	A secondary index patient_email-index is used to retrieve scans for the signed-in patient.
Lambda AccessDenied errors during testing	Lambda execution role did not always have permission for DynamoDB or S3 operations.	IAM permissions were added for required actions such as PutItem, Scan, Query, UpdateItem, and S3 presigned access.

Table 6.7. Cloud implementation challenges and applied fixes.

Figure 6.3. Screenshot evidence gallery showing AWS resources and backend testing during development.

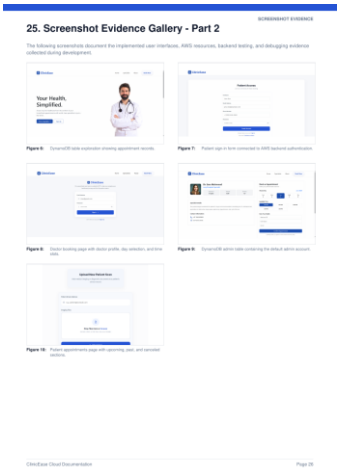


Figure 6.4. Screenshot evidence gallery showing frontend pages connected to the backend workflows.

6.8 Lambda Function Implementation

This section lists the Lambda functions used in the cloud backend. The functions are intentionally separated by business task. This makes the system easier to test, debug, and extend. Each function returns JSON and includes CORS headers so the web frontend can communicate with it from the browser.

Important Design Choice

The system uses small focused Lambda functions instead of one large backend file. This allows each endpoint to be tested independently and reduces the risk that a change in the scan workflow breaks the booking workflow.

1) clinicease-patient-signup

1) clinicease-patient-signup

```
import json, boto3, hashlib
from datetime import datetime, timezone
dynamodb = boto3.resource('dynamodb')
patients_table = dynamodb.Table('ClinicEasePatients')
def response(code, body):
    return {
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
```

```
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
        },
        'body': json.dumps(body)
    }
def hash_password(password):
    return hashlib.sha256(password.encode('utf-8')).hexdigest()
def lambda_handler(event, context):
    try:
        if event.get('httpMethod') == 'OPTIONS':
            return response(200, {'message': 'CORS OK'})
        body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event
        full_name = body.get('full_name', '').strip()
        email = body.get('email', '').strip().lower()
        phone = body.get('phone', '').strip()
        password = body.get('password', '').strip()
        if not full_name or not email or not phone or not password:
            return response(400, {'message': 'All fields are required.'})
        patients_table.put_item(Item={
            'email': email,
            'full_name': full_name,
            'phone': phone,
            'password_hash': hash_password(password),
```

```
            'role': 'patient',
            'created_at': datetime.now(timezone.utc).isoformat()
        })
        return response(200, {'message': 'Patient account created successfully.'})
    except Exception as e:
        print('Signup error:', str(e))
        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda receives new patient registration data from the signup page, validates the fields, hashes the password, and stores the patient record in ClinicEasePatients. It converts the frontend form into a persistent cloud account record.

2) clinicease-patient-signin

2) clinicease-patient-signin

```
import json, boto3, hashlib
dynamodb = boto3.resource('dynamodb')
patients_table = dynamodb.Table('ClinicEasePatients')
admins_table = dynamodb.Table('ClinicEaseAdmins')
def response(code, body):
    return {
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
```

```
        },
        'body': json.dumps(body)
    }
def hash_password(password):
    return hashlib.sha256(password.encode('utf-8')).hexdigest()
def lambda_handler(event, context):
    try:
        if event.get('httpMethod') == 'OPTIONS':
            return response(200, {'message': 'CORS OK'})
        body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event
        email = body.get('email', '').strip().lower()
        password = body.get('password', '').strip()
        password_hash = hash_password(password)
        admin_res = admins_table.get_item(Key={'email': email})
        admin = admin_res.get('Item')
        if admin and admin.get('password_hash') == password_hash:
            return response(200, {'message': 'Admin signed in.', 'user': {
                'email': admin['email'], 'name': admin.get('full_name', 'Admin'), 'role': 'admin'
            }})
        patient_res = patients_table.get_item(Key={'email': email})
        patient = patient_res.get('Item')
        if patient and patient.get('password_hash') == password_hash:
            return response(200, {'message': 'Patient signed in.', 'user': {
```

```
                'email': patient['email'], 'name': patient.get('full_name', ''), 'role': 'patient'
            }})
        return response(401, {'message': 'Invalid email or password.'})
    except Exception as e:
        print('Signin error:', str(e))
        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda checks both ClinicEaseAdmins and ClinicEasePatients. It returns a role-based user object so the frontend can redirect admins to the dashboard and patients to the normal portal.

3) clinicease-book-appointment

3) clinicease-book-appointment

```
import json, boto3, uuid
from datetime import datetime, timezone
dynamodb = boto3.resource('dynamodb')
appointments_table = dynamodb.Table('ClinicEaseAppointments')
def response(code, body):
    return {
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
        },
        'body': json.dumps(body)
```

```
    }
def lambda_handler(event, context):
    try:
        if event.get('httpMethod') == 'OPTIONS':
            return response(200, {'message': 'CORS OK'})
        body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event
        required = ['patient_name', 'patient_email', 'national_id', 'doctor_name',
                    'doctor_specialty', 'day', 'time', 'appointment_date']
        missing = [field for field in required if not body.get(field)]
        if missing:
            return response(400, {'message': 'Missing required fields.', 'missing': missing})
        appointment = {
            'appointment_id': 'apt-' + str(uuid.uuid4())[:12],
            'patient_name': body['patient_name'],
            'patient_email': body['patient_email'].lower(),
            'national_id': body['national_id'],
            'doctor_name': body['doctor_name'],
            'doctor_specialty': body['doctor_specialty'],
            'day': body['day'],
            'time': body['time'],
            'appointment_date': body['appointment_date'],
            'status': 'Pending',
            'created_at': datetime.now(timezone.utc).isoformat()
```

```
        }
        appointments_table.put_item(Item=appointment)
        return response(200, {'message': 'Appointment booked successfully.', 'appointment': appointment})
    except Exception as e:
        print('Booking error:', str(e))
        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda is the core appointment-booking service. It validates patient and doctor data, generates appointment_id, stores appointment_date and appointment status, and returns the created appointment to the confirmation page.

4) clinicease-get-my-appointments

4) clinicease-get-my-appointments

```
import json, boto3
from boto3.dynamodb.conditions import Attr
dynamodb = boto3.resource('dynamodb')
appointments_table = dynamodb.Table('ClinicEaseAppointments')
def response(code, body):
    return {
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
        },
```

```
        'body': json.dumps(body)
    }
def lambda_handler(event, context):
    try:
        if event.get('httpMethod') == 'OPTIONS':
            return response(200, {'message': 'CORS OK'})
        body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event
        patient_email = body.get('patient_email', '').strip().lower()
        if not patient_email:
            return response(400, {'message': 'patient_email is required.'})
        result = appointments_table.scan(FilterExpression=Attr('patient_email').eq(patient_email))
        appointments = result.get('Items', [])
        appointments.sort(key=lambda x: x.get('appointment_date', ''), reverse=True)
        return response(200, {'appointments': appointments})
    except Exception as e:
        print('Get my appointments error:', str(e))
        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda retrieves the signed-in patient appointments using patient_email. The frontend then separates them into upcoming, past, and cancelled sections based on date and status.

5) clinicease-get-all-appointments

```
5) clinicease-get-all-appointments
import json, boto3
dynamodb = boto3.resource('dynamodb')
```

```
appointments_table = dynamodb.Table('ClinicEaseAppointments')

def response(code, body):
    return {
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
        },
        'body': json.dumps(body)
    }
def lambda_handler(event, context):
    try:
        if event.get('httpMethod') == 'OPTIONS':
            return response(200, {'message': 'CORS OK'})
        items = []
        result = appointments_table.scan()
        items.extend(result.get('Items', []))
        while 'LastEvaluatedKey' in result:
            result = appointments_table.scan(ExclusiveStartKey=result['LastEvaluatedKey'])
            items.extend(result.get('Items', []))
        items.sort(key=lambda x: x.get('created_at', ''), reverse=True)
        return response(200, {'appointments': items})
```

```
    except Exception as e:
        print('Get all appointments error:', str(e))
        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda is used by the admin dashboard. It scans the appointments table and returns all records so the admin can monitor the clinic activity and patient flow.

6) clinicease-update-appointment-status

6) clinicease-update-appointment-status

```
import json, boto3
from datetime import datetime, timezone
dynamodb = boto3.resource('dynamodb')
appointments_table = dynamodb.Table('ClinicEaseAppointments')
def response(code, body):
    return {
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
        },
        'body': json.dumps(body)
    }
def lambda_handler(event, context):
    try:
```

```
    if event.get('httpMethod') == 'OPTIONS':
        return response(200, {'message': 'CORS OK'})
    body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event
    appointment_id = body.get('appointment_id')
    status = body.get('status')
    allowed = ['Pending', 'Confirmed', 'Cancelled', 'Completed']
    if not appointment_id or status not in allowed:
        return response(400, {'message': 'appointment_id and valid status are required.'})
    result = appointments_table.update_item(
        Key={'appointment_id': appointment_id},
        UpdateExpression='SET #s = :s, updated_at = :u',
        ExpressionAttributeNames={'#s': 'status'},
        ExpressionAttributeValues={':s': status, ':u': datetime.now(timezone.utc).isoformat()},
        ReturnValues='ALL_NEW'
    )
    return response(200, {'message': 'Appointment status updated.', 'appointment':
result.get('Attributes')})
except Exception as e:
    print('Status update error:', str(e))
    return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda allows the admin to change appointment status. The status update immediately affects the patient-facing appointment page and the admin dashboard.

7) clinicease-generate-scan-upload-url

7) clinicease-generate-scan-upload-url

```
import json, boto3, uuid

s3 = boto3.client('s3')

BUCKET_NAME = 'clinicease-scans-mohamed'

def response(code, body):

    return {

        'statusCode': code,

        'headers': {

            'Access-Control-Allow-Origin': '*',

            'Access-Control-Allow-Headers': 'Content-Type',

            'Access-Control-Allow-Methods': 'OPTIONS,POST'

        },

        'body': json.dumps(body)

    }

def lambda_handler(event, context):

    try:

        if event.get('httpMethod') == 'OPTIONS':

            return response(200, {'message': 'CORS OK'})

        body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event

        patient_email = body.get('patient_email', '').strip().lower()

        file_name = body.get('file_name', '').strip()

        content_type = body.get('content_type', 'application/octet-stream')

        if not patient_email or not file_name:
```

```
            return response(400, {'message': 'patient_email and file_name are required.'})

        scan_id = 'scan-' + str(uuid.uuid4())

        safe_email = patient_email.replace('@', '_at_').replace('.', '_')

        file_key = f'scans/{safe_email}/{scan_id}-{file_name}'

        upload_url = s3.generate_presigned_url(

            'put_object',

            Params={'Bucket': BUCKET_NAME, 'Key': file_key, 'ContentType': content_type},

            ExpiresIn=900

        )

        return response(200, {'scan_id': scan_id, 'file_key': file_key, 'upload_url': upload_url})

    except Exception as e:

        print('Generate upload URL error:', str(e))

        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: This Lambda creates a temporary S3 presigned upload URL. The file is uploaded directly from the browser to S3, so large medical files do not pass through Lambda.

8) clinicease-save-scan-metadata

8) clinicease-save-scan-metadata

```
import json, boto3

from datetime import datetime, timezone

dynamodb = boto3.resource('dynamodb')

scans_table = dynamodb.Table('ClinicEaseScans')

def response(code, body):

    return {
```

```
        'statusCode': code,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Access-Control-Allow-Headers': 'Content-Type',
            'Access-Control-Allow-Methods': 'OPTIONS,POST'
        },
        'body': json.dumps(body)
    }
def lambda_handler(event, context):
    try:
        if event.get('httpMethod') == 'OPTIONS':
            return response(200, {'message': 'CORS OK'})
        body = json.loads(event.get('body', '{}')) if isinstance(event.get('body'), str) else event
        required = ['scan_id', 'patient_email', 'uploaded_by', 'file_name', 'file_key', 'content_type']
        missing = [field for field in required if not body.get(field)]
        if missing:
            return response(400, {'message': 'Missing required fields.', 'missing': missing})
        item = {
            'scan_id': body['scan_id'],
            'patient_email': body['patient_email'].lower(),
            'uploaded_by': body['uploaded_by'],
            'file_name': body['file_name'],
            'file_key': body['file_key'],
```

```
            'content_type': body['content_type'],
            'uploaded_at': datetime.now(timezone.utc).isoformat()
        }
        scans_table.put_item(Item=item)
        return response(200, {'message': 'Scan metadata saved successfully.', 'scan': item})
    except Exception as e:
        print('Save scan metadata error:', str(e))
        return response(500, {'message': 'Internal server error', 'error': str(e)})
```

Description: After the file upload succeeds, this Lambda stores scan metadata in ClinicEaseScans. The metadata connects the patient email with the S3 file key and display information.

6.9 Frontend Integration with the Cloud

The website pages are connected to the AWS backend through JavaScript fetch requests. The booking page uses hospital.js, the admin dashboard uses dashboard.js, and the patient scan viewer uses patient-scans.js. These files act as the bridge between user actions and AWS cloud services.

Frontend File	Cloud Responsibility	Important Notes
hospital.js	Books appointments and loads patient appointment history.	Uses /book-appointment and /my-appointments. It must generate the correct appointment_date to avoid wrong Past/Upcoming classification.
dashboard.js	Loads all appointments, updates status, and handles scan upload.	Uses get-all-appointments, update-appointment-status, generate-scan-upload-url, and save-scan-metadata.
patient-scans.js	Loads patient scans and requests download URLs.	Reads currentUser from localStorage and sends patient_email to get-patient-scans.
signup.html / signin.html	Creates and authenticates patient accounts.	Uses patient-signup and patient-signin endpoints.

Table 6.8. Frontend files and their cloud responsibilities.

6.10 Cloud Cost Estimation for Real Deployment

The following estimate assumes a small clinic or hospital department deployment. Actual pricing depends on region, traffic, file size, logging volume, and free tier eligibility. The estimate is useful for judging whether cloud deployment is financially suitable compared with buying and maintaining a local server.

Assumption	Estimated Monthly Volume
Patient, admin, scan, and booking API calls	100,000 requests / month
Sensor readings from 10 ESP32 devices	432,000 readings / month (one reading per minute per device)
Total API Gateway requests	About 532,000 requests / month
DynamoDB writes	About 450,000 writes / month
DynamoDB reads	About 200,000 reads / month
Stored scan files	20 GB in S3 Standard
Scan upload/download requests	1,000–5,000 requests / month
Lambda memory/duration	128–256 MB, short execution time

Table 6.9. Cost estimation assumptions for a pilot clinic deployment.

AWS Service	Pricing Basis Used	Estimated Monthly Cost
API Gateway REST API	About 532k requests. At roughly $3.50 per million REST API calls after free tier.	$0.00–$1.90
AWS Lambda	About 532k executions. Free tier often covers 1M requests and 400k GB-seconds; otherwise requests are low cost.	$0.00–$0.30
DynamoDB On-Demand Writes	About 450k writes at about $0.625 per million writes.	$0.28
DynamoDB On-Demand Reads	About 200k reads at about $0.125 per million reads.	$0.03
DynamoDB Storage	Small structured data. First 25 GB can be covered by free tier in many accounts.	$0.00–$1.00
Amazon S3 Storage	20 GB scan storage at about $0.023 per GB-month.	$0.46
S3 Requests	PUT/GET/LIST requests for scan upload and download.	$0.05–$0.20
CloudWatch Logs	Basic logs for Lambda debugging and monitoring.	$1.00–$3.00
Estimated Total	Serverless backend + scan storage + monitoring.	About $3–$8 / month after optimization

Table 6.10. Estimated monthly AWS cost by service.

Estimated annual cloud cost: approximately $36–$96 per year for the pilot usage level above. In the first 12 months, some usage may be lower because AWS Free Tier can cover a portion of API Gateway, Lambda, and DynamoDB usage depending on account eligibility.

6.11 Cloud Compared with a Local Server

A local server may look simple at the beginning, but it requires hardware purchase, continuous power, storage backup, networking, updates, security hardening, and maintenance. The cloud approach avoids upfront hardware cost and shifts the backend to managed services that scale based on usage.

Cost Item	Estimated Local Server Monthly Cost	Reason
Server hardware amortization	$25–$40	A small server paid over three years.
UPS and storage backup	$10–$20	Power protection and backup disks.
Electricity and cooling	$8–$20	Server must run continuously.
Static IP / stronger internet plan	$15–$30	Needed for reliable external access.
Maintenance and security updates	$60–$120	Admin time for backups, patching, firewall rules, failures, and monitoring.
Unexpected hardware replacement	$10–$30	Disk, power supply, network equipment, or downtime risk.
Estimated Total	$128–$260 / month	Typical practical cost range for a small local deployment.

Table 6.11. Estimated monthly local server cost.

Comparison Point	Cloud Backend	Local Server
Upfront cost	Very low, pay-as-you-go.	Requires server, UPS, storage, and setup.
Monthly cost for pilot	About $3–$8.	About $128–$260.
Annual cost for pilot	About $36–$96.	About $1,536–$3,120.
Maintenance	AWS manages infrastructure.	Team must manage OS, backups, hardware, and security.
Scalability	Scales with requests and storage.	Limited by purchased hardware.
Availability	Uses AWS managed regional infrastructure.	Depends on local power, internet, and hardware health.
Security	IAM, private S3, managed HTTPS, presigned URLs.	Requires manual firewall, certificates, updates, and backup strategy.

Table 6.12. Cloud backend versus local server.

Estimated Saving

Using the cloud for this project can reduce early deployment cost from roughly $128–$260 per month to about $3–$8 per month. This means the cloud can save approximately $120–$250 per month, or about 90%–97% of early infrastructure cost, while also reducing maintenance effort and downtime risk.

6.12 Why Cloud Was the Better Choice for ClinicEase

- The project is a prototype that needs real backend behavior without the cost and complexity of physical servers.
- Serverless services are suitable because traffic is event-based: signup, signin, booking, status update, scan upload, and sensor readings.
- S3 presigned URLs are more secure and scalable for medical scan files than storing files on a local machine.
- DynamoDB removes database server administration and fits the project data model well.
- Lambda makes the backend modular and easy to explain, test, and improve.
- Cloud deployment allows future mobile, web, and hardware components to communicate with the same backend from anywhere.

6.13 Future Cloud Improvements

The current cloud backend is suitable for an academic prototype. For a real hospital deployment, the following improvements can make the system stronger, safer, and more professional.

Future Improvement	How It Strengthens the Project
Amazon Cognito	Provides professional user authentication, password recovery, MFA, and token-based access instead of frontend localStorage sessions.
Single API Gateway Structure	Combines the two current APIs into one organized API with clear stages such as dev and prod.
CloudFront + Route 53 + ACM	Provides a custom HTTPS domain, faster website delivery, and managed SSL certificates.
IAM Least Privilege Policies	Limits every Lambda to only the DynamoDB table or S3 action it needs.
S3 Encryption and Lifecycle Rules	Encrypts scans at rest and moves old scans to cheaper storage classes automatically.
DynamoDB Point-in-Time Recovery	Protects patient and appointment data against accidental deletion or data corruption.
CloudWatch Dashboards and Alarms	Monitors API errors, Lambda failures, high latency, and unusual traffic.
AWS WAF	Adds protection against common web attacks and abusive traffic.
SNS / SES Notifications	Sends appointment confirmations, cancellation notices, or sensor alerts to patients and admins.
AWS IoT Core	Provides a stronger production path for ESP32/Arduino devices instead of direct HTTP posting only.
CI/CD Pipeline	Automates deployment of frontend and Lambda code using GitHub Actions or AWS CodePipeline.
Backup and Audit Logging	Keeps traceable logs for admin actions and healthcare data access.

Table 6.13. Recommended future cloud improvements.

6.14 Chapter Summary

The cloud and backend solution is one of the most important parts of the ClinicEase project because it connects the website, admin dashboard, scan management, and future hardware readings into one shared system. AWS API Gateway, Lambda, DynamoDB, and S3 were used to implement the backend without managing physical servers. The solution supports patient registration, authentication, appointment booking, admin management, medical scan upload/download, and future sensor data integration. Compared with a local server, the serverless cloud approach is cheaper for early deployment, easier to maintain, more scalable, and more suitable for a graduation prototype that can be expanded into a real healthcare platform.

6.15 Pricing Sources Used for Estimation

- AWS API Gateway public pricing page: REST API requests are billed per million requests after any eligible free tier.

- AWS Lambda public pricing page: request price is per million requests, with free tier for eligible accounts.
- AWS DynamoDB public pricing page: on-demand reads, writes, and storage are billed based on request units and stored GB.
- AWS S3 public pricing page: object storage and requests are billed based on storage class, stored GB, and request volume.

Chapter 7: System Integration, Results and Challenges

7.1 Integration between Network, Web, Mobile and Cloud

The project becomes valuable when the separate services are viewed as one integrated solution. The network design provides the infrastructure of the hospital. The web system provides browser-based access for guests, patients, and admins. The mobile application provides patient booking and notification access through smartphones. The cloud backend stores and manages the shared data that connects these interfaces. Together, these parts form a smart hospital software ecosystem.

In a future complete deployment, hospital devices and hardware systems can send data to the same cloud platform through secure APIs. The admin dashboard can then display operational data, appointment statistics, scan records, and possibly sensor readings. This creates a path from the current software prototype to a full smart hospital platform.

7.2 Testing Results

Area Tested	Result
Network Connectivity	VLANs, DHCP, wireless, IoT components, and routing were configured and verified in Packet Tracer.
Network Redundancy	HSRP and STP provided redundancy and loop prevention; root bridge configuration improved stability.
Web Workflow	Guest, patient, and admin routes were implemented through separate pages and dashboard views.
Mobile Workflow	Flutter screens supported booking flow, state management, localization, and notifications.
Cloud Authentication	Sign-up and sign-in workflows created records and returned patient/admin roles.
Cloud Appointments	Appointment records were stored and retrieved from DynamoDB.
Medical Scans	S3 presigned URLs supported direct file upload and secure download.

7.3 Implementation Challenges

The project faced several challenges. In the network part, Packet Tracer visual indicators sometimes did not match actual STP states, so CLI verification was required. Root bridge election also caused an access switch to become root until bridge priorities were manually configured. In the web and cloud part, CORS errors appeared during API and S3 testing, so CORS headers and bucket rules had to be adjusted. In early web versions, localStorage caused data to remain browser-specific, which motivated the cloud migration. For medical scans, direct file upload through Lambda was avoided by using presigned URLs. In the mobile part, responsive design, state separation, and bilingual support required careful architecture.

7.4 Achieved Outcomes

- A complete smart hospital software concept was designed and documented.
- The network infrastructure was simulated with redundancy, segmentation, security, wireless access, and IoT monitoring.
- A role-based healthcare website was developed with guest, patient, and admin flows.
- A mobile booking application was designed and documented using Flutter architecture.
- AWS serverless backend services were implemented for patient accounts, appointments, and scans.

- The project demonstrated how multiple software engineering services can be delivered by one integrated team.

Chapter 8: Conclusion and Future Work

Conclusion

Integrated Software Engineering Solutions successfully demonstrates how several software engineering services can be combined into one practical smart hospital prototype. The project does not focus on only one isolated system; instead, it presents network design, web development, mobile application development, cloud backend, and integration as one complete solution. This reflects the real needs of modern organizations that require connected digital systems rather than independent tools.

The network solution shows how a hospital can be supported by a structured, secure, and scalable infrastructure. VLANs isolate departments, HSRP improves gateway availability, STP prevents loops, DHCP automates addressing, and security features protect management and access ports. The web and mobile solutions show how patients and administrators can interact with the healthcare system through clear interfaces. The cloud solution shows how backend functionality can be implemented using AWS serverless services, DynamoDB tables, S3 storage, and presigned URLs.

The project achieved its educational and practical goal by proving that one team can design, implement, test, and document several integrated software services for a realistic healthcare case study. The result is a strong prototype that can be extended into a production-ready smart hospital system.

Recommendations for Future Work

- Replace prototype authentication with AWS Cognito or another production identity provider.
- Add token-based authorization and role-based access control for all backend endpoints.
- Use CloudFront and a custom HTTPS domain for secure and optimized website delivery.
- Improve dashboard analytics with charts, filtering, search, and real-time notifications.

- Connect hardware sensors and embedded systems to the cloud backend through secure APIs.
- Add audit logging for scan uploads, downloads, and appointment status changes.
- Expand the hospital network to include more floors, departments, and branch connectivity.
- Improve mobile integration with real backend APIs, push notifications, and patient profile management.
- Add database indexes and optimized access patterns for large-scale appointment and patient data.

Bibliography

[1] AWS Documentation. Amazon S3 User Guide. Amazon Web Services. Available online: https://docs.aws.amazon.com/s3/ [Accessed 2026].

[2] AWS Documentation. AWS Lambda Developer Guide. Amazon Web Services. Available online: https://docs.aws.amazon.com/lambda/ [Accessed 2026].

[3] AWS Documentation. Amazon API Gateway Developer Guide. Amazon Web Services. Available online: https://docs.aws.amazon.com/apigateway/ [Accessed 2026].

[4] AWS Documentation. Amazon DynamoDB Developer Guide. Amazon Web Services. Available online: https://docs.aws.amazon.com/dynamodb/ [Accessed 2026].

[5] Cisco Systems. Cisco Packet Tracer and Networking Academy learning resources. Available online: https://www.netacad.com/ [Accessed 2026].

[6] Cisco Systems. Spanning Tree Protocol, VLAN, HSRP, and switching configuration resources. Available online: https://www.cisco.com/ [Accessed 2026].

[7] Flutter Documentation. Flutter development framework. Available online: https://docs.flutter.dev/ [Accessed 2026].

[8] Dart Documentation. Dart programming language. Available online: https://dart.dev/guides [Accessed 2026].

[9] United Nations. Sustainable Development Goals. Available online: https://sdgs.un.org/goals [Accessed 2026].

[10] Egypt Vision 2030. Sustainable Development Strategy: Egypt Vision 2030. Available online: https://mped.gov.eg/ [Accessed 2026].

[11] AIET. Final Year Project Report Template 2025–2026. Alexandria Higher Institute of Engineering & Technology.

[12] Project Team Documentation. ClinicEase Cloud Documentation, ClinicEase Booking App Documentation, Hospital Network Project Documentation, and Integrated Engineering Web System Report. Internal graduation project reports, 2025–2026.

Appendix

Appendix A: Project Team Members

Name	ID	Department	Role
Ali Ayman Mohamed Gamal Eldeen	21-0-6163	ECE	Network, Web Developer & Team Leader
Mohamed Elshazly AbdElaziz Mobarak	22-1-11156	ECE	Network & Cloud Solutions
Kyrellos Hany Saied Fahem	21-0-6301	ECE	Network, Cloud Solutions
Marwan Samy Mostafa Elzwawy	21-0-6101	CE	Mobile Application
Rewan Alaa Abd Elmaksoud	21-0-8202	ECE	Network
Eman Ahmed Ghaith	21-0-7836	ECE	Network
Alaa Reda Mohammad Abo Mehana	21-0-7633	ECE	Network
Rowaida Ahmed Abd El-Wahab	21-0-6220	ECE	Network
Aya Ahmed Khattab	21-0-6014	ECE	Network & Web Developer
Mohamed Ahmed Mohamed Aly	21-0-6295	ECE	Cloud Solutions

Appendix B: Main AWS Resources

- ClinicEasePatients DynamoDB table
- ClinicEaseAdmins DynamoDB table
- ClinicEaseAppointments DynamoDB table
- ClinicEaseScans DynamoDB table

- S3 bucket for static website hosting
- S3 bucket for medical scan files
- API Gateway endpoints for authentication, appointments, dashboard, and scans
- Python Lambda functions for backend logic

Appendix C: Main Network Configuration Concepts

- HSRP virtual gateway between R1 and R2
- Core switch inter-VLAN routing using SVIs
- Trunk links between core, distribution, and access switches
- DHCP helper addresses on VLAN interfaces
- SSH secure remote access
- Port security and sticky MAC addresses
- DHCP snooping trusted uplinks
- STP root primary and secondary configuration

Appendix D: Summarized Network CLI Commands

This appendix keeps only the essential command concepts needed to prove the implemented configuration. Repetitive port-by-port listings and repeated screenshots were removed because the network chapter already explains the design and includes the main visual evidence.

D.1 HSRP Gateway Redundancy

R1/R2 interface G0/0/0

```
 ip address 192.168.1.2 / 192.168.1.3 255.255.255.0
 standby 1 ip 192.168.1.1
 standby 1 priority 105      ! on R1
 standby 1 preempt
 ip route 192.168.0.0 255.255.0.0 192.168.1.5
```

D.2 Core Switch Inter-VLAN Routing

```
ip routing
interface Vlan10
 ip address 192.168.10.1 255.255.255.0
 ip helper-address 192.168.100.10
interface Vlan20 / Vlan30 / Vlan40 / Vlan50
 use the same pattern with their VLAN gateways
```

D.3 VLANs, Trunks, and Access Links

```
vlan 10,20,30,40,50,100
interface G1/0/24
 switchport mode trunk
interface F0/x
```

switchport mode access

switchport access vlan [10/20/30/40/50]

D.4 DHCP Snooping and Port Security

ip dhcp snooping

ip dhcp snooping vlan 10,20,30,40,50

interface [trusted uplink]

ip dhcp snooping trust

interface [access port]

switchport port-security

switchport port-security mac-address sticky

switchport port-security violation shutdown

D.5 SSH Secure Management

ip domain-name project.local

crypto key generate rsa modulus 1024

username admin secret 1234

line vty 0 4

transport input ssh

login local

show ip ssh